Machine Learning

Unit 1

- Introduction to Machine Learning
- Concept of the learning task, inductive learning, and the concepts of hypothesis space
- Introduction to different types of machine learning approaches, examples of machine learning applications
- Different types of learning: supervised learning, unsupervised learning, reinforcement learning.
- Feature Extraction- Setting up your machine learning platform: training, validation and testing, overfitting and under-fitting, different types of error calculation.

What is Learning?

"To gain **knowledge** or **understanding** of, or **skill** in **by study**, **instruction** or **experience**"

- Learning a set of new facts.
- Learning HOW to do something .
- Improving ability of something already learned.

Machine Learning

- Term popularized by Arthur Samuel in 1959.
- Sub-part of Artificial Intelligence that uses data to teach machines.
- Field of study that gives computers the ability to learn without being explicitly programmed.
- Building machines that automatically learn from experience.
- Machine learning use the theory of statistics in building mathematical models, because the core task is making inference from a sample.

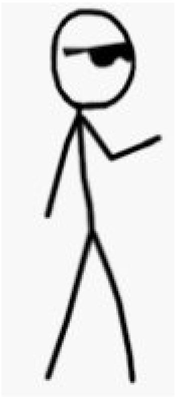
Machine Learning

Humans learn from experience and adapt their actions for future tasks.

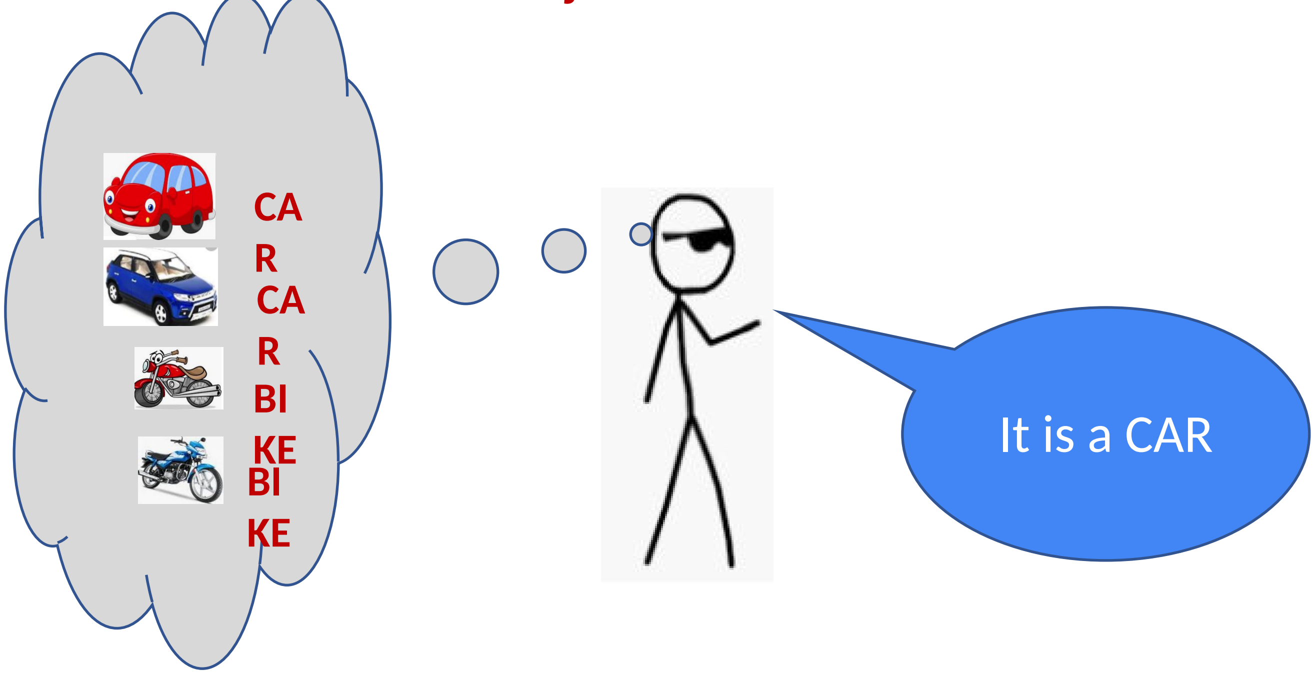
Can machines adapt their behavior using experience?

Human can learn from past experience
and make decision of its own

What is this object?



What is this object?

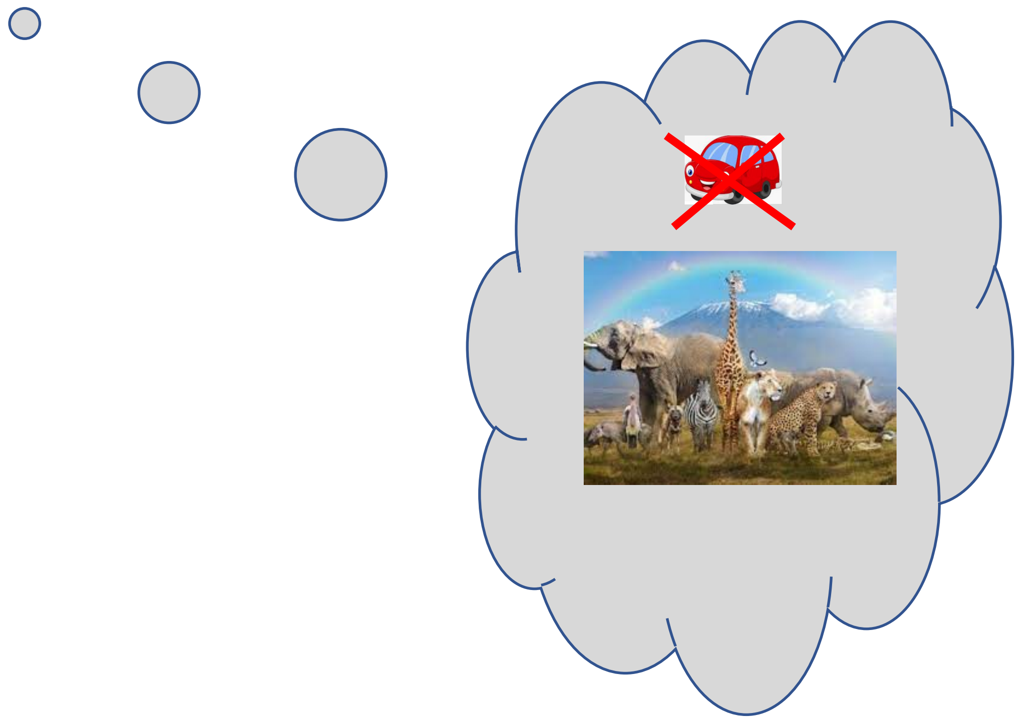


Let us ask the same question to him

What is this object?

Let us ask the same question to him

What is this object?



[But, he is a human being. He can observe and learn]

Let us make him learn

show
him

Let us make him learn

show
him

CAR

CAR

BIKE

BIKE

Let us ask the same question now

What is this object?

CAR

CAR

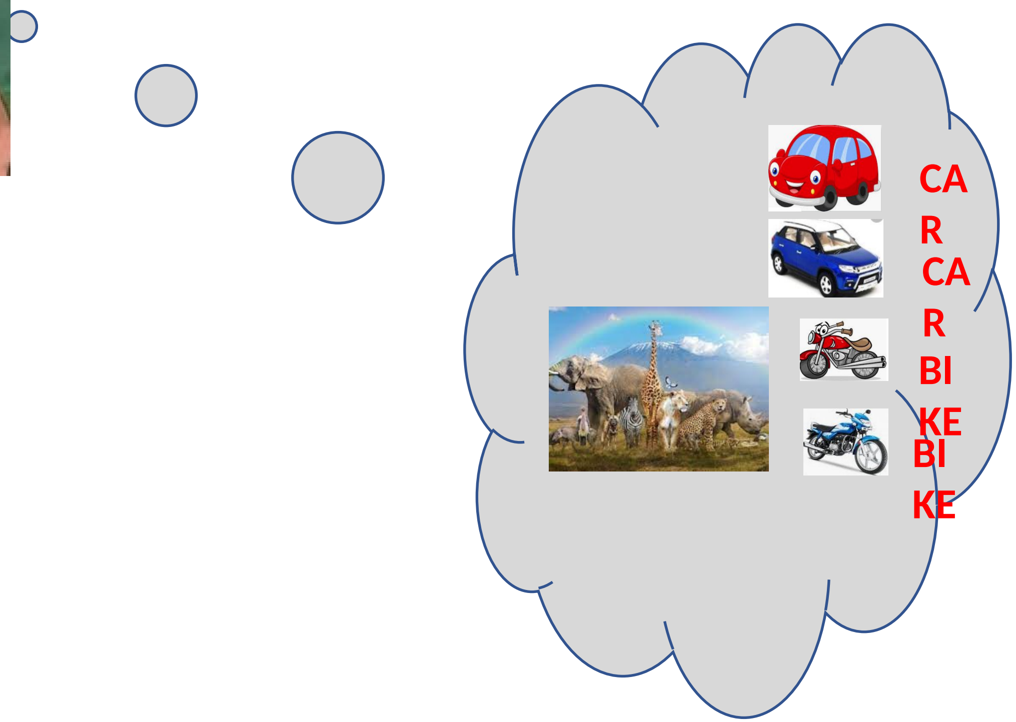
BIKE

BIKE

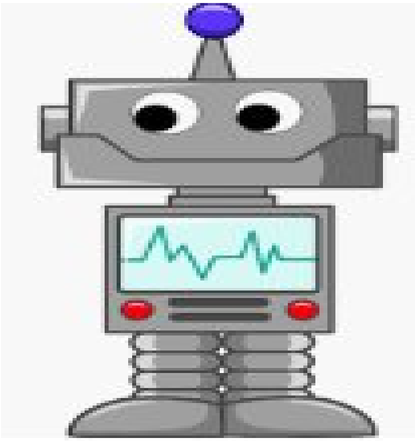
Past experience

Let us ask the same question now

What is this object?



What about a Machine ?



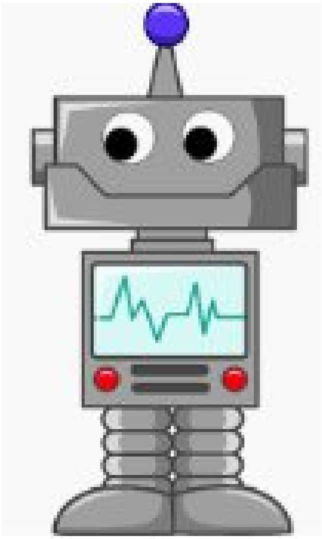
Machines follow instructions

It can not take decision of its own

What about a Machine ?

We can ask a machine

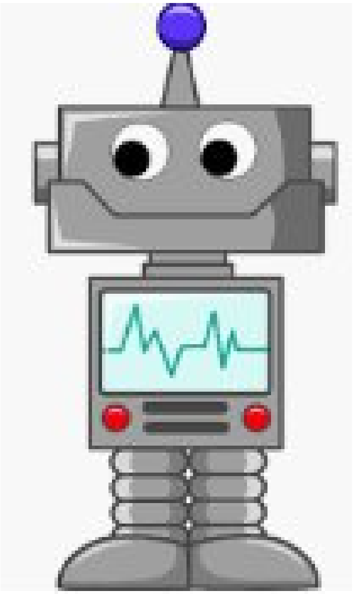
- To perform an arithmetic operations such as
  - Addition
  - Multiplication
  - Division



Machines follow instructions

What about a Machine ?

- Comparison
- Print
- Plotting a chart



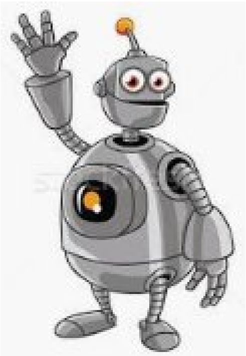
Machines follow instructions

But, we can ask a machine to make a decision of its own

What is Machine Learning?

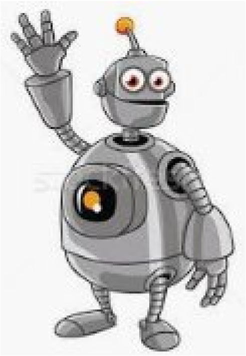
We want a machine to act like a human

What is Machine Learning?



[to identify this object.]

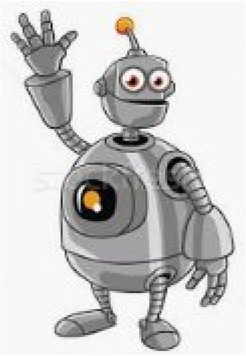
What is Machine Learning?



[predict the price in future]

Price in 2025?

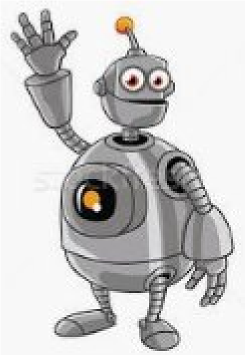
What is Machine Learning?



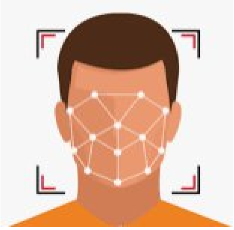
I ~~made~~ met him yesterday

[Natural Language understand, and correct grammar]

What is Machine Learning?



[Recognize Faces]

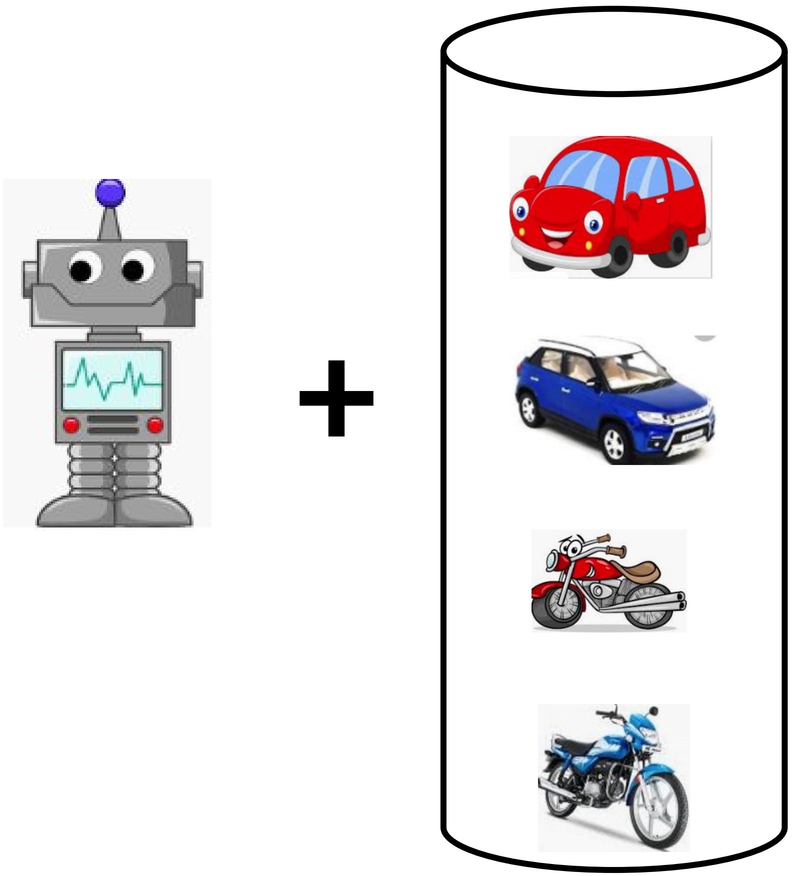


What is Machine Learning?

Just like, what we did to human,

we need to provide experience to the machine.

What is Machine Learning?

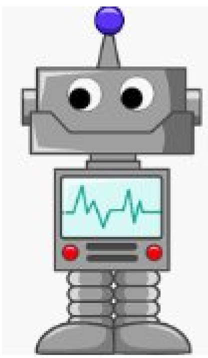


Dataset

This what we called as Data or Training dataset

So, we first need to provide training dataset to the machine

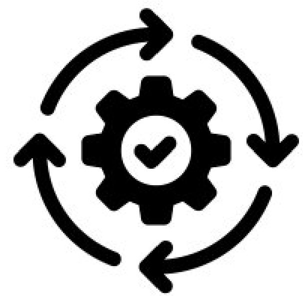
What is Machine Learning?



+

Dataset

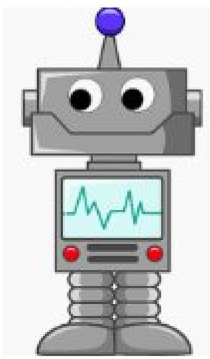
+



[Then, devise algorithms and execute programs on the data

With respect to the underlying target tasks]

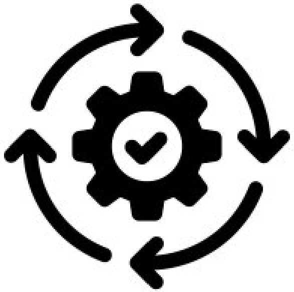
What is Machine Learning?



+

Datase
t

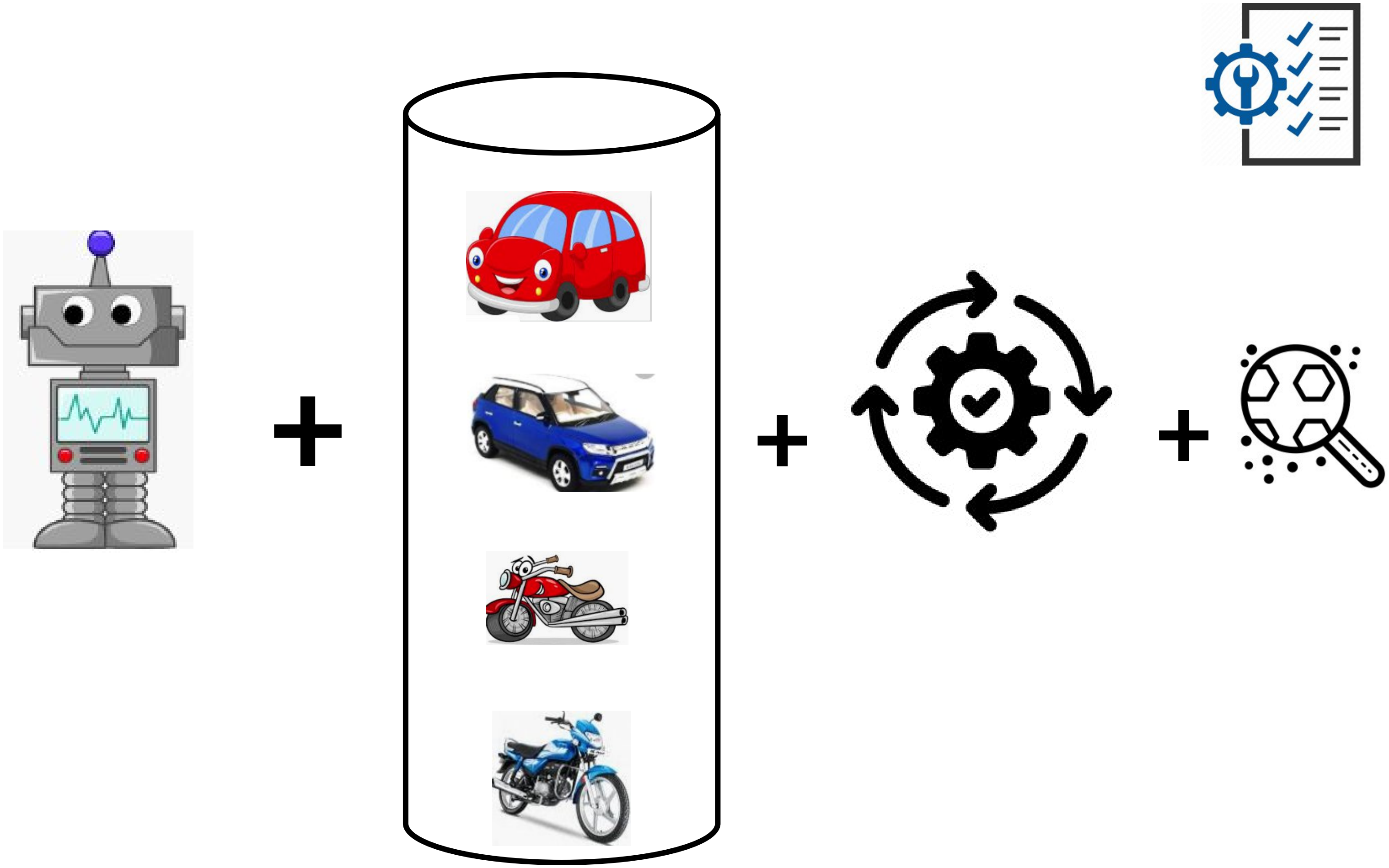
+



+

[Then, using the programs, Identify required rules]

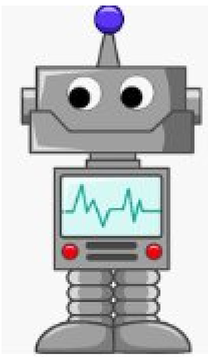
What is Machine Learning?



Datase
t

[extract required patterns]

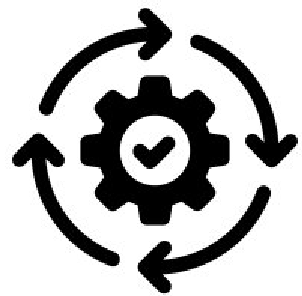
What is Machine Learning?



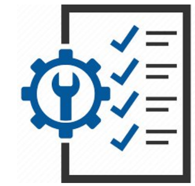
+

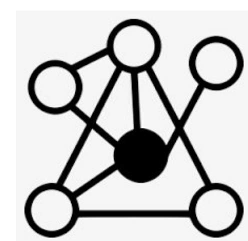
**Datase
t**

+



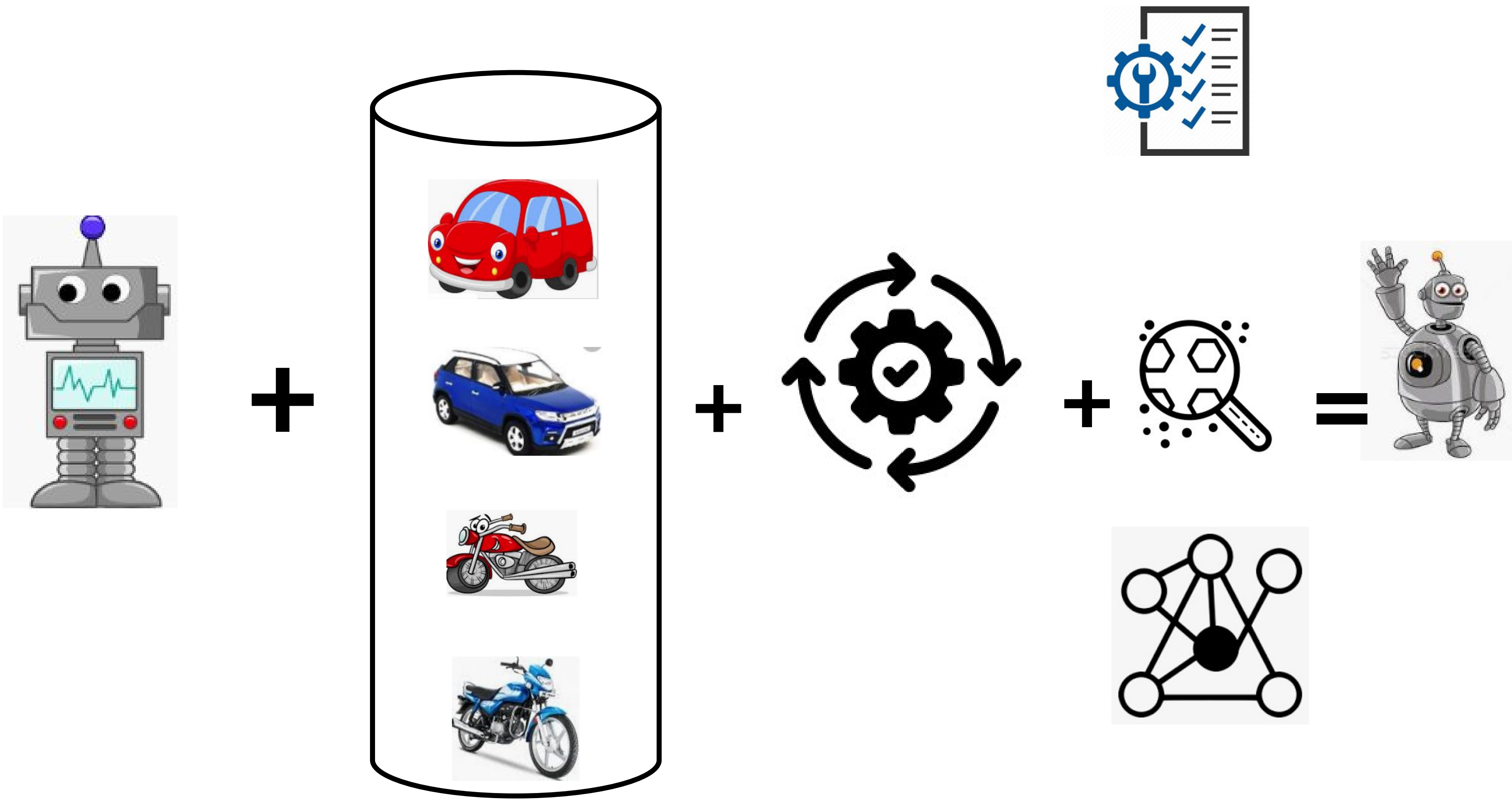
+





[Identify relations]

What is Machine Learning?



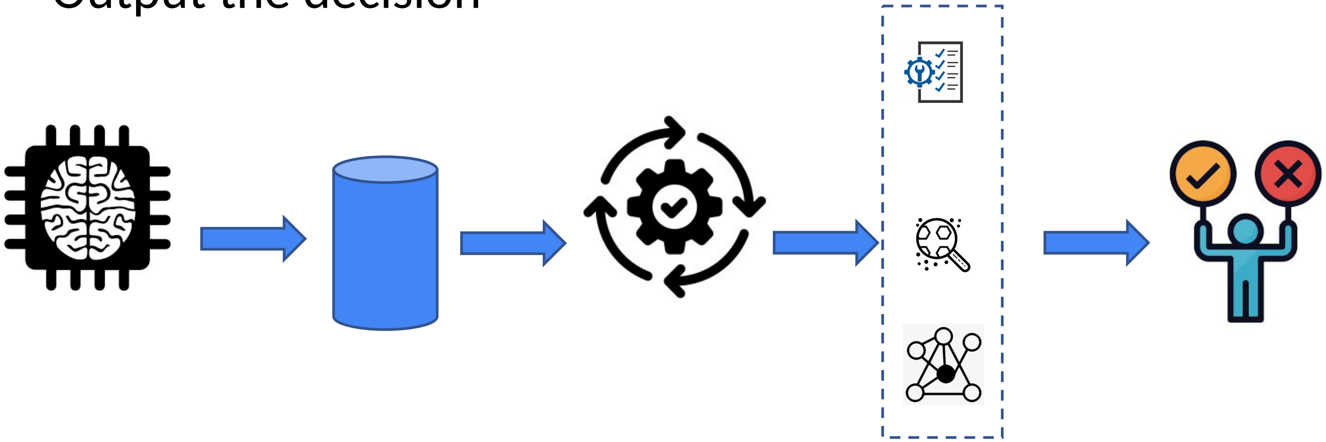
Datase
t

[So that machine can derive inferences from the data]

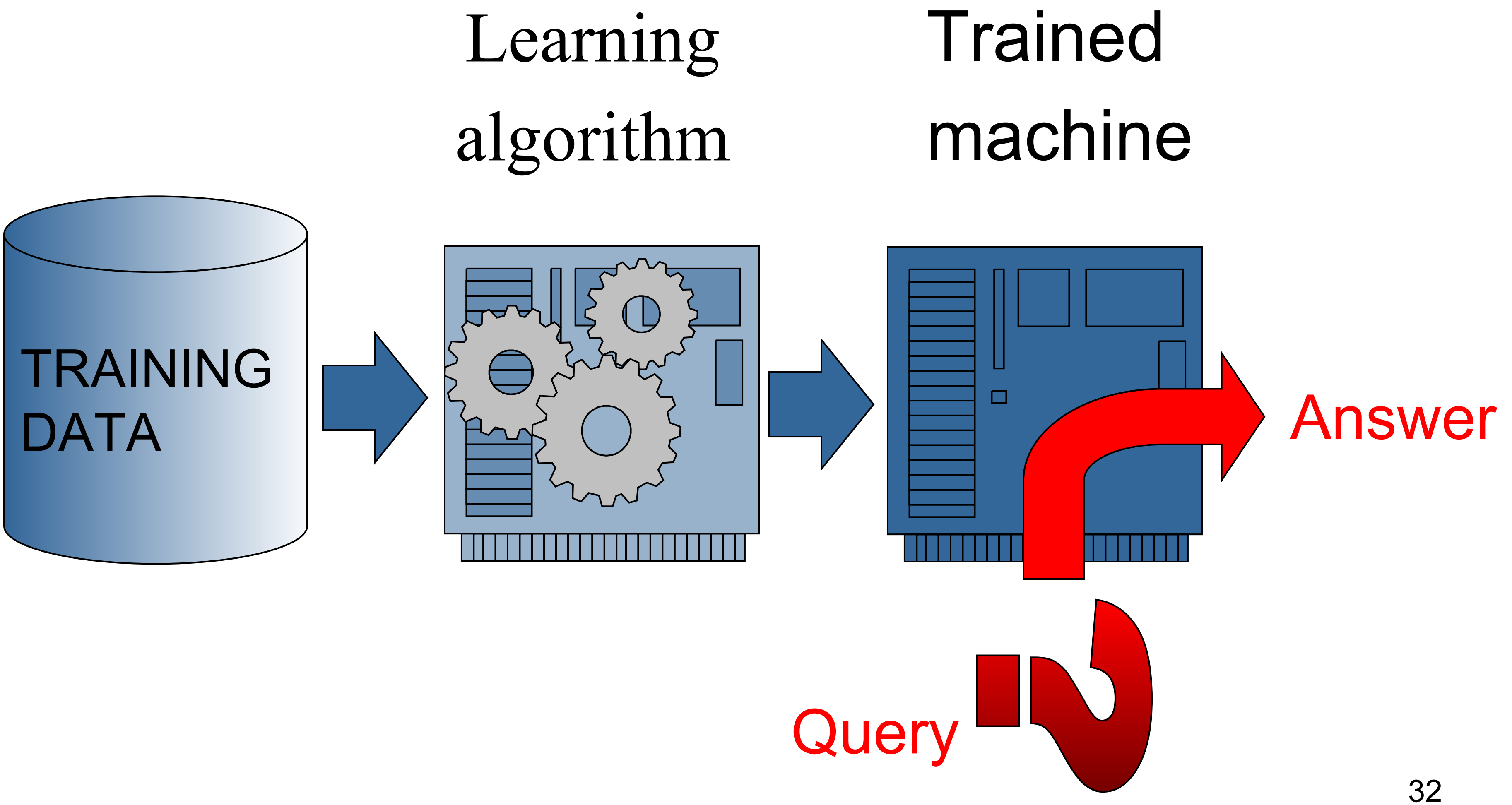
In summary, what is machine learning?

Given a machine learning problem

- Identify and create the appropriate dataset
- Perform computation to learn
  - Required rules, pattern and relations
- Output the decision

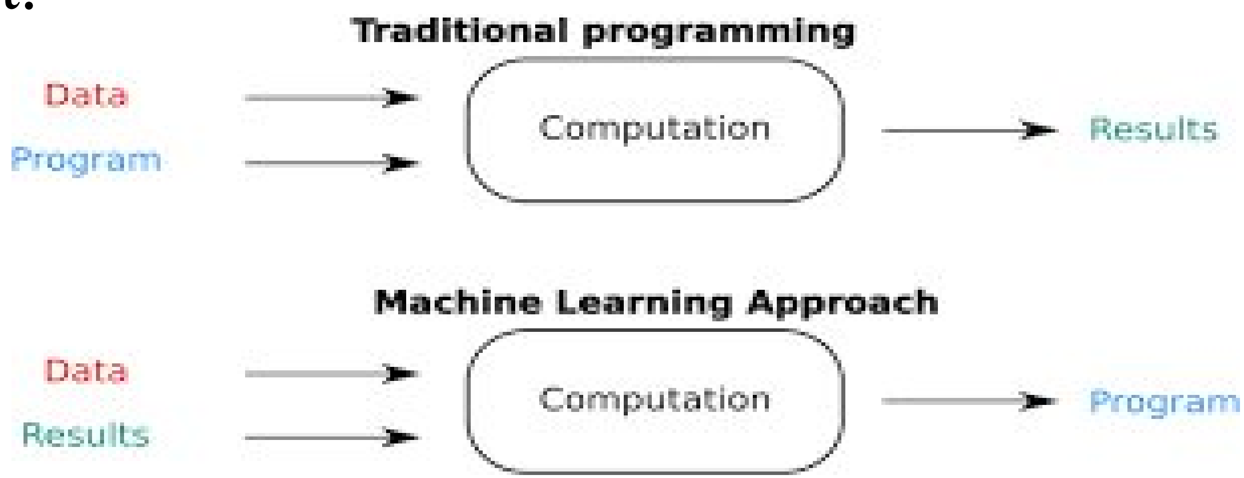


What is Machine Learning?



Machine Learning

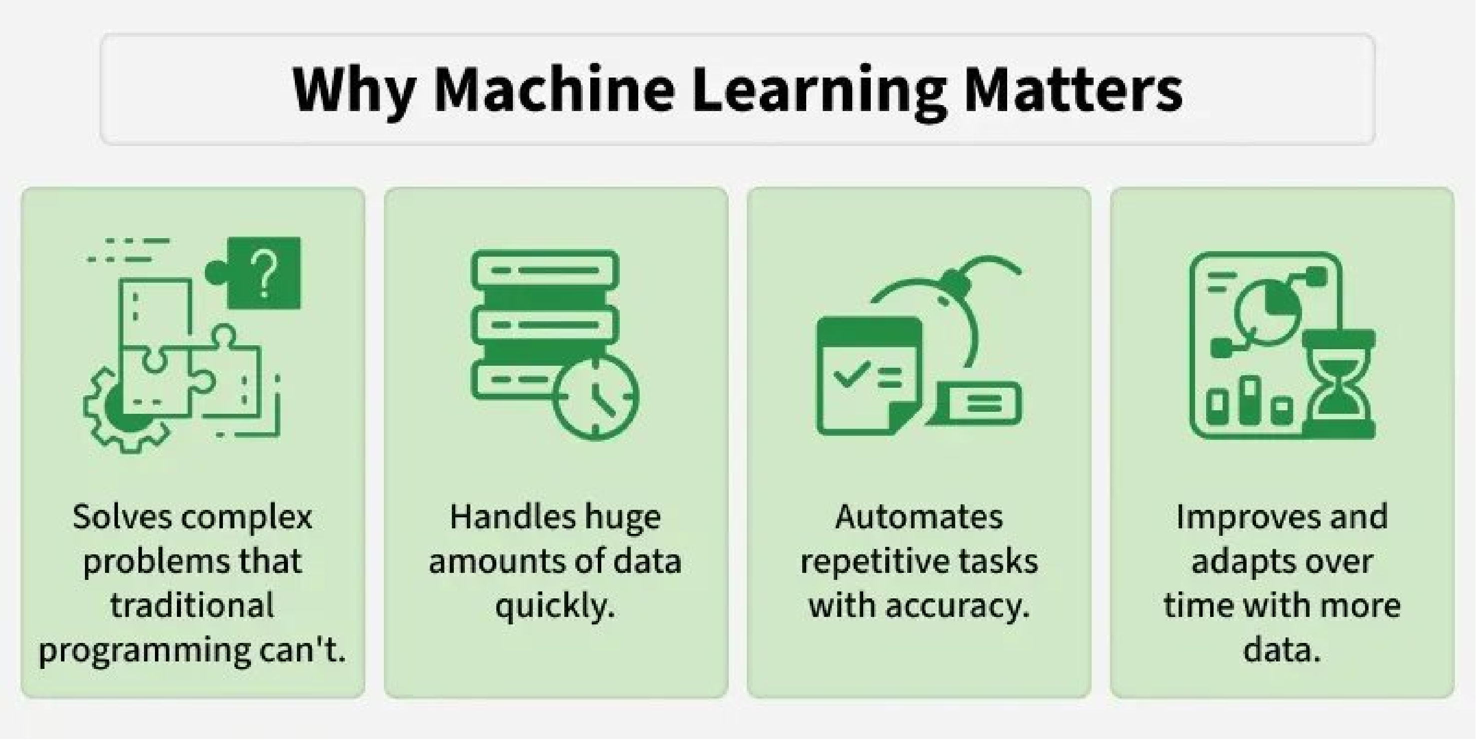
- Machine Learning system learns from historical data (training data) , builds prediction models and whenever it receives new data, predicts the output for it.



Need for Machine Learning

Machine Learning is important because traditional programming cannot handle complex tasks or large amounts of data efficiently.

ML overcomes this by learning from data and making predictions without fixed rules.



Need for Machine Learning

1. Solving Complex Business Problems

Traditional programming struggles with tasks like language understanding and medical diagnosis. ML learns from data and predicts outcomes easily.

Examples:

- Image and speech recognition in healthcare.
- Language translation and sentiment analysis.

Need for Machine Learning

2. Handling Large Volumes of Data

The internet generates huge amounts of data every day. Machine Learning processes and analyzes this data quickly by providing valuable insights and real time predictions.

Examples:

- Fraud detection in financial transactions.
- Personalized feed recommendations on Facebook and Instagram from billions of interactions.

Need for Machine Learning

3. Automate Repetitive Tasks

ML automates time consuming, repetitive tasks with high accuracy hence reducing manual work and errors.

Examples:

- Gmail filtering spam emails automatically.
- Chatbots handling order tracking and password resets.
- Automating large scale invoice analysis for key insights.

Need for Machine Learning

4. Personalized User Experience

ML enhances user experience by tailoring recommendations to individual preferences. It analyze user behavior to deliver highly relevant content.

Examples:

- Netflix suggesting movies and TV shows based on our viewing history.
- E-commerce sites recommending products we're likely to buy.

Need for Machine Learning

5. Self Improvement in Performance

ML models evolve and improve with more data helps in making them smarter over time. They adapt to user behavior and increase their performance.

Examples:

- Voice assistants like Siri and Alexa learning our preferences and accents.
- Search engines refining results based on user interaction.
- Self driving cars improving decisions using millions of miles of driving data.

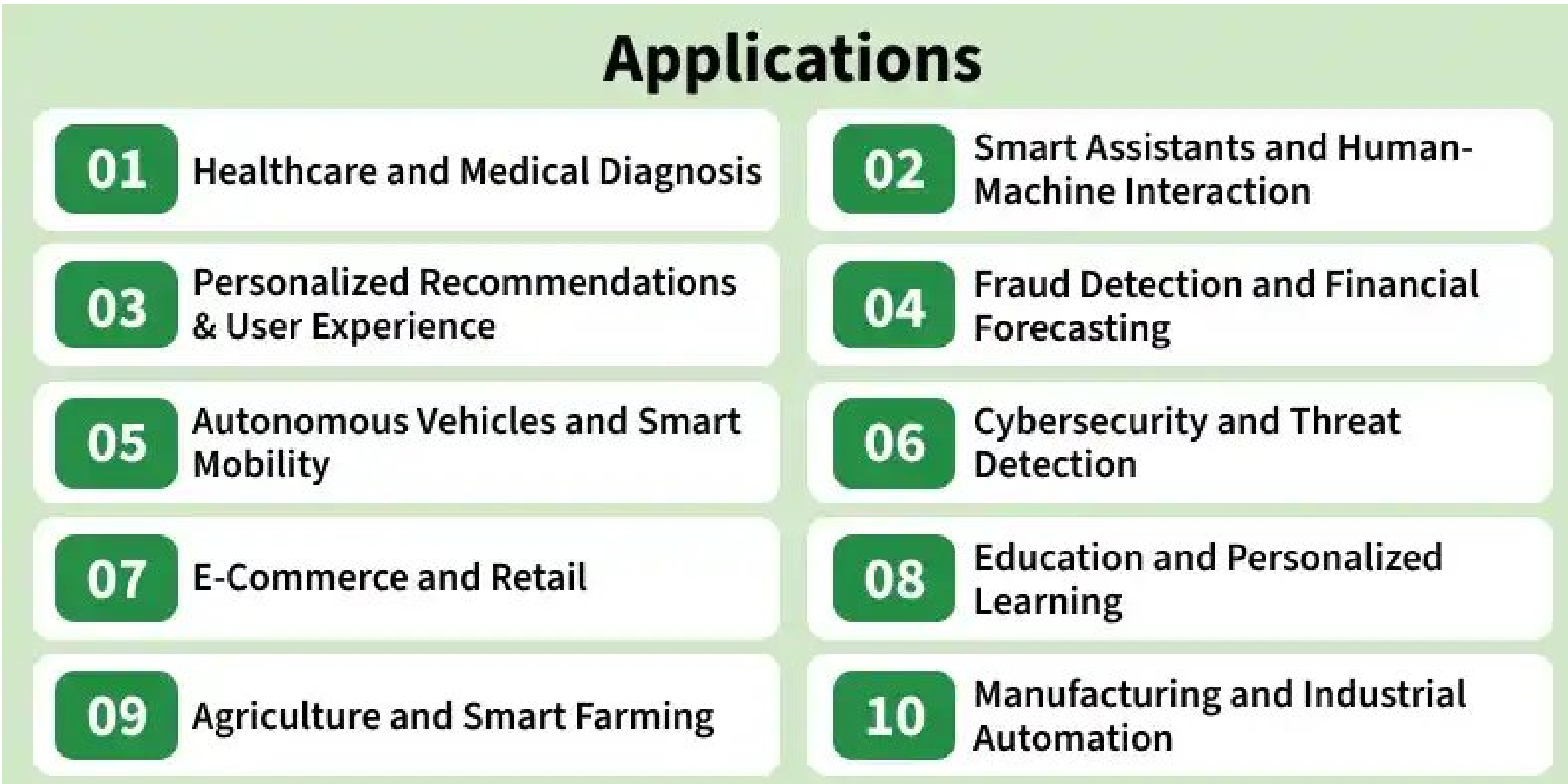
Advantages of ML

1. Machine Learning improves processes by automating tasks and extracting insights from data, making systems smarter and more efficient.
2. Automates repetitive tasks and improves productivity
3. Finds patterns in large data for better decisions
4. Provides customized recommendations and experiences
5. Enables robots and systems to perform complex tasks accurately

Disadvantages of ML

1. Application specific algorithms.
2. Real world problems have too many variables and sensors might be too noisy.
3. Computational complexity.
4. ML models learn from training data and if the data is biased, model's decisions can be unfair so it's important to select and monitor data carefully.
5. Since it depends on large amounts of data, there is a risk of sensitive information being exposed so protecting privacy is important.
6. Complex ML models can be difficult to understand which makes it difficult to explain why they make certain decisions. This can affect trust and accountability.
7. Automation may replace some jobs so retraining and helping workers learn new skills is important to adapt to these changes.

Applications of Machine Learning



Applications of Machine Learning

1. Healthcare and Medical Diagnosis

ML algorithms can analyze large volumes of patient data, medical scans and genetic information to aid in diagnosis and treatment.

- **Disease Detection**: ML models are used to identify diseases like cancer, pneumonia and Parkinson's from medical images. They often achieve accuracy comparable to or better than human doctors.
- **Predictive Analytics**: By analyzing patient history and symptoms, models can predict the risk of certain diseases or potential complications.
- **Drug Discovery**: ML accelerates the drug development process by predicting how different compounds will interact, reducing the time and cost of research.

Applications of Machine Learning

2. Smart Assistants and Human-Machine Interaction

Virtual assistants systems rely on natural language processing (NLP) and speech recognition to understand commands and respond intelligently.

- **Voice Assistants**: Tools like Siri, Alexa and Google Assistant convert spoken input into actionable commands.
- **Voice Search & Transcription**: ML enables users to perform hands-free web searches and get transcription during meetings or phone calls.
- **Chatbots**: Businesses use AI-powered chatbots for 24/7 customer support, helping resolve queries faster and more efficiently.

Applications of Machine Learning

3. Personalized Recommendations and User Experience

Modern digital platforms uses personalization which is done by using **recommender systems. M**achine learning models analyze user behavior to deliver relevant content, improving engagement and satisfaction.

- **Streaming Platforms**: Netflix and Spotify suggest shows and songs based on your watching or listening history.
- **E-commerce**: Sites like Amazon recommend products tailored to your preferences, browsing patterns and past purchases.
- **Social Media**: Algorithms curate content feeds, prioritize posts and suggest friends or pages.

These systems use techniques like collaborative filtering and content-based filtering to create personalized digital experiences.

Applications of Machine Learning

4. Fraud Detection and Financial Forecasting

In finance, vast sums of money move digitally and machine learning plays an important role in fraud detection and market analysis.

- **Transaction Monitoring:** Banks use ML models to detect unusual spending behavior and flag suspicious transactions.
- **Loan Risk Assessment:** Credit scoring models analyze customer profiles and predict the likelihood of default.
- **Stock Market Prediction:** ML is used to analyze historical stock data and forecast price movements. Stock markets are complex, algorithmic trading uses these predictions for better decision-making.

Applications of Machine Learning

5. Autonomous Vehicles and Smart Mobility

Self-driving vehicles use ML to understand their environment, navigate safely and make immediate decisions.

- **Computer Vision**: Recognizing lanes, pedestrians, traffic signals and obstacles.
- **Sensor Fusion**: Combining data from cameras, LiDAR and radar for a 360-degree view.
- **Behavior Prediction**: Anticipating how other drivers or pedestrians may act.

Autonomous vehicles are capable of operating with minimal human input. Beyond cars, ML is also being used in traffic optimization, smart navigation systems and predictive maintenance in transportation.

Applications of Machine Learning

6. Cybersecurity and Threat Detection

Machine learning helps identify cyber threats, detect suspicious activities, and improve digital security systems by analyzing network behavior and user activity patterns.

- **Spam and Phishing Detection:** ML filters detect spam emails, phishing links, and malicious attachments.
- **Intrusion Detection:** Security systems monitor unusual network activity to identify cyberattacks in real-time.
- **Malware Detection:** ML models analyze software behavior to detect malicious programs and ransomware.

Applications of Machine Learning

7. E-Commerce and Retail

E-commerce platforms use machine learning to optimize customer experience, inventory management, and sales strategies.

- **Demand Forecasting:** Predicts product demand to improve inventory planning and reduce shortages.
- **Dynamic Pricing:** Adjusts product prices based on demand, competition, and customer behavior.
- **Customer Segmentation:** Groups customers based on interests and purchasing behavior for targeted marketing.

Applications of Machine Learning

8. Education and Personalized Learning

Machine learning improves learning experiences by adapting educational content to individual student needs.

- **Personalized Learning:** Recommends customized study materials and learning paths.
- **Automated Grading:** Evaluates quizzes, assignments, and objective answers automatically.
- **Student Performance Prediction:** Identifies students who may need additional academic support.

Applications of Machine Learning

9. Agriculture and Smart Farming

ML-powered systems help farmers improve productivity, monitor crops, and optimize agricultural resources.

- **Crop Disease Detection:** Identifies plant diseases using image analysis and computer vision.
- **Yield Prediction:** Predicts crop production using weather and soil data.
- **Smart Irrigation:** Optimizes water usage based on environmental conditions and crop requirements.

Applications of Machine Learning

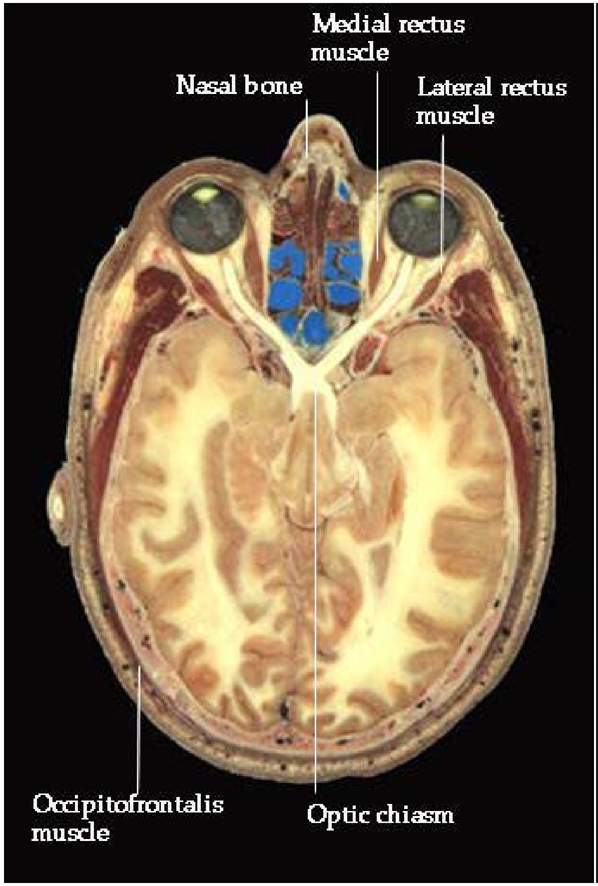
10. Manufacturing and Industrial Automation

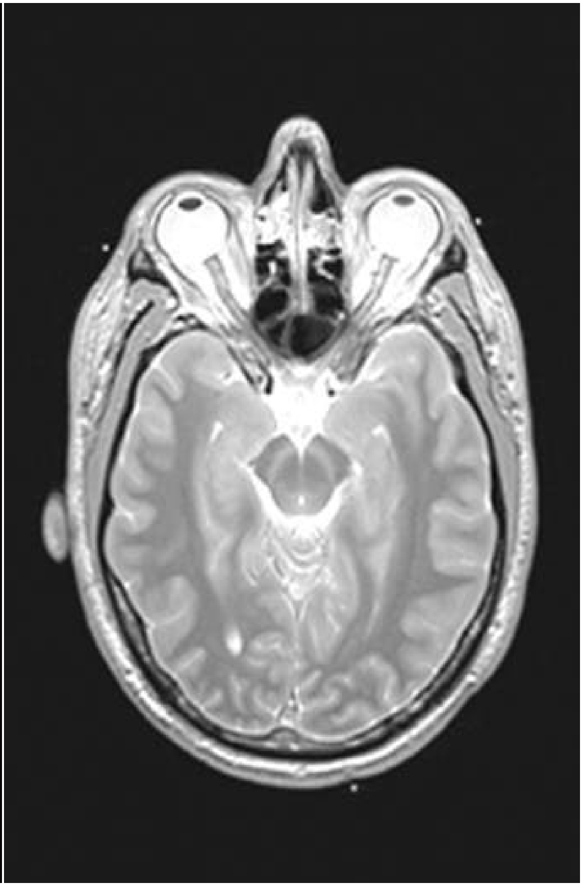
Machine learning enhances industrial efficiency through predictive analysis and automation.

- **Predictive Maintenance:** Detects equipment issues before failures occur.
- **Quality Inspection:** Uses computer vision to identify defects in manufactured products.
- **Robotics Automation:** Enables intelligent robots to perform repetitive industrial tasks efficiently

Drug discovery

Medical diagnosis





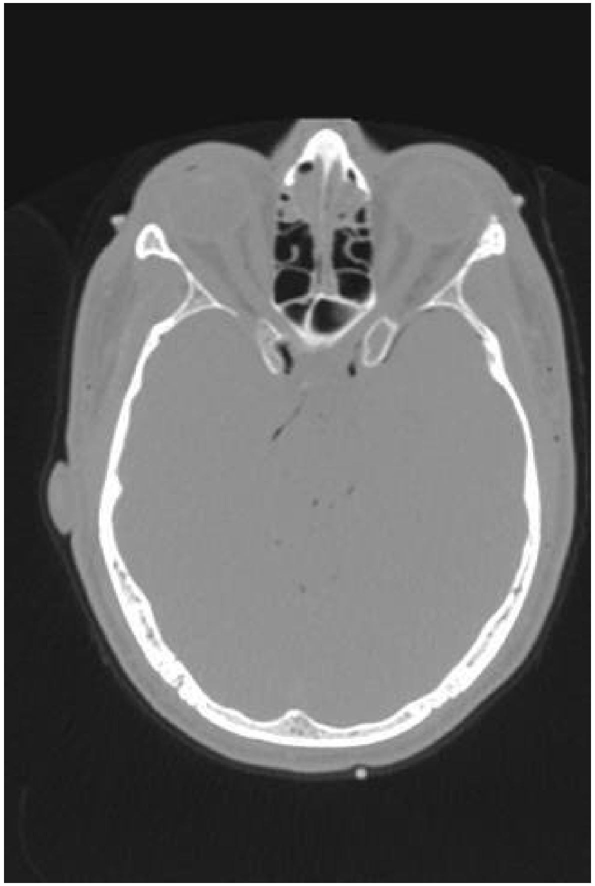
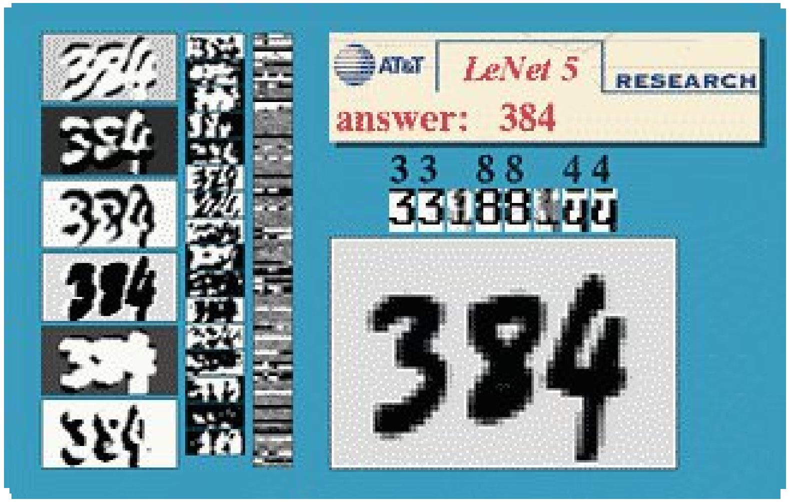
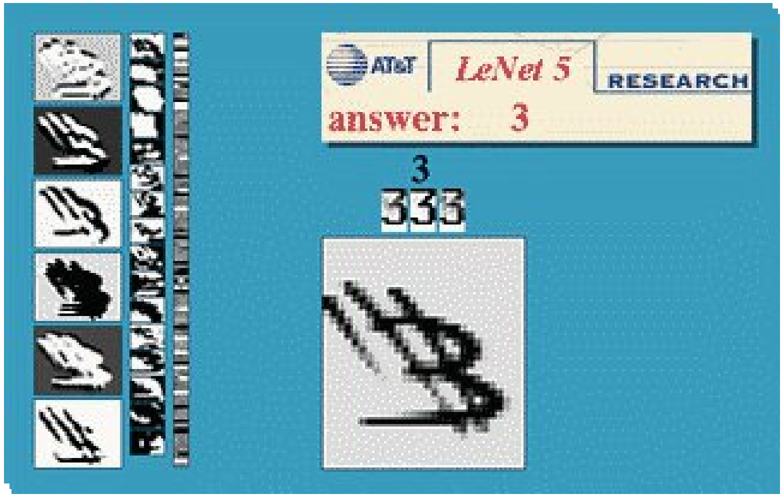


Photo MRI CT

Iris verification

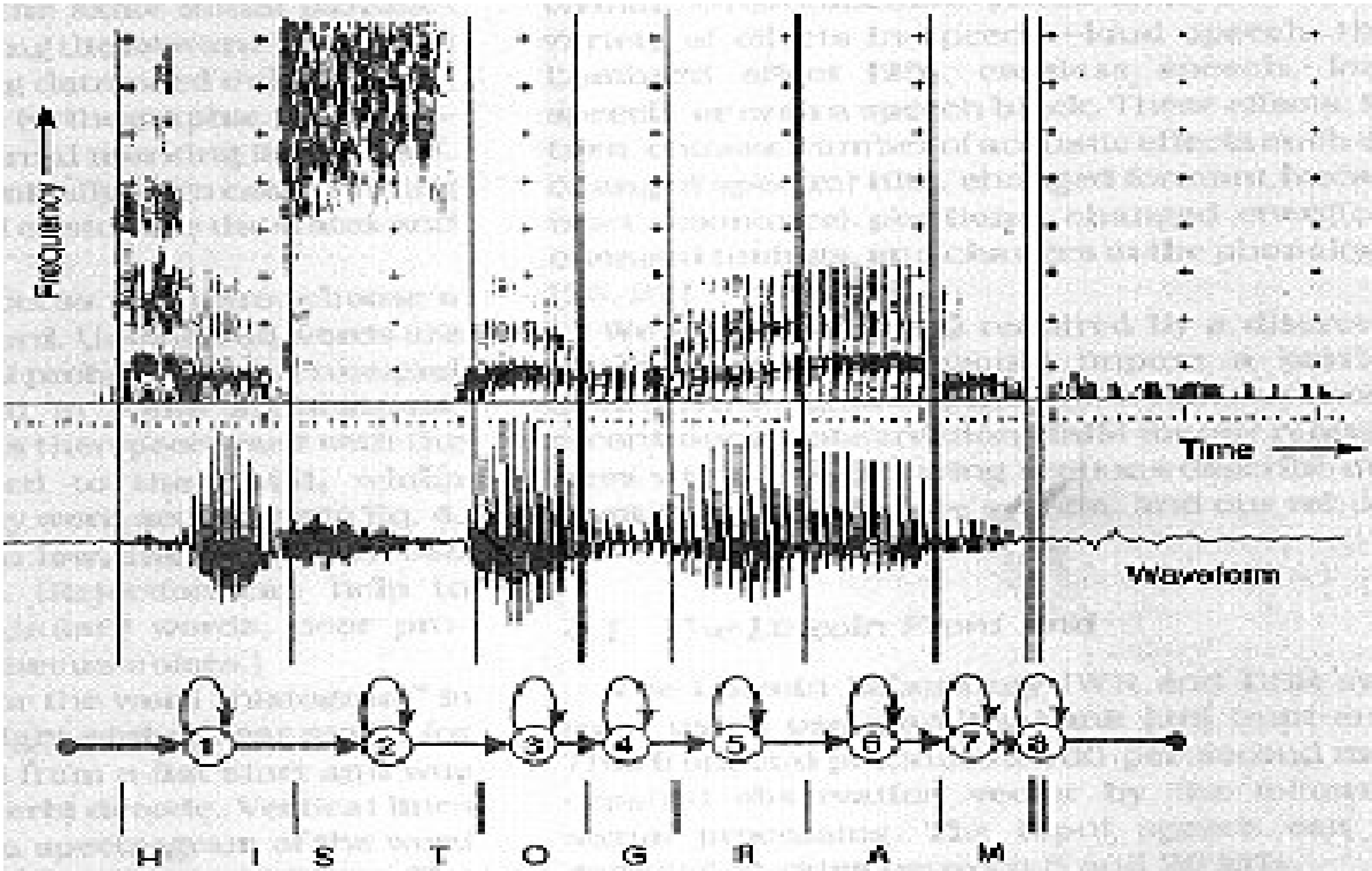
Hand-written digits



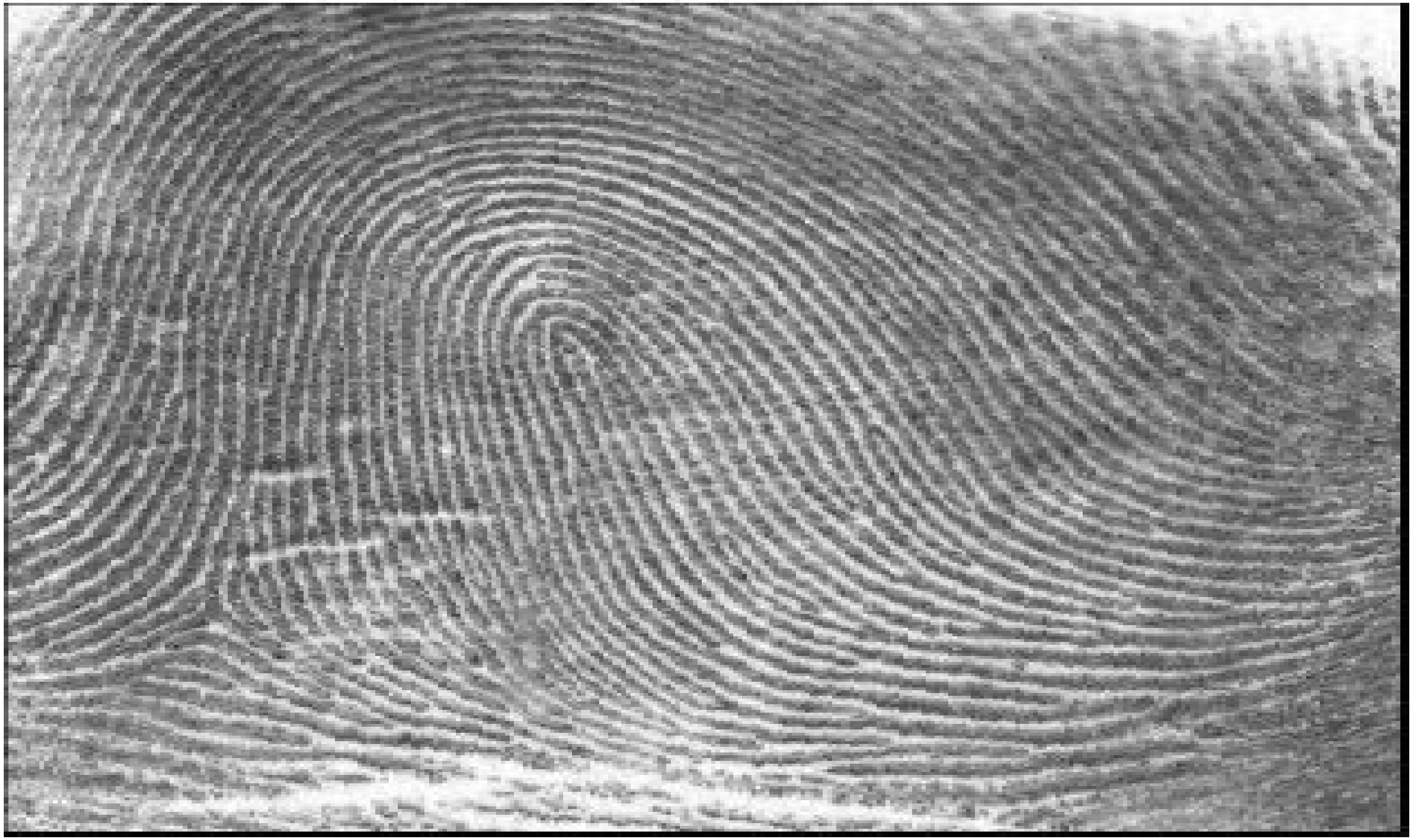


Radar Imaging

Speech Recognition

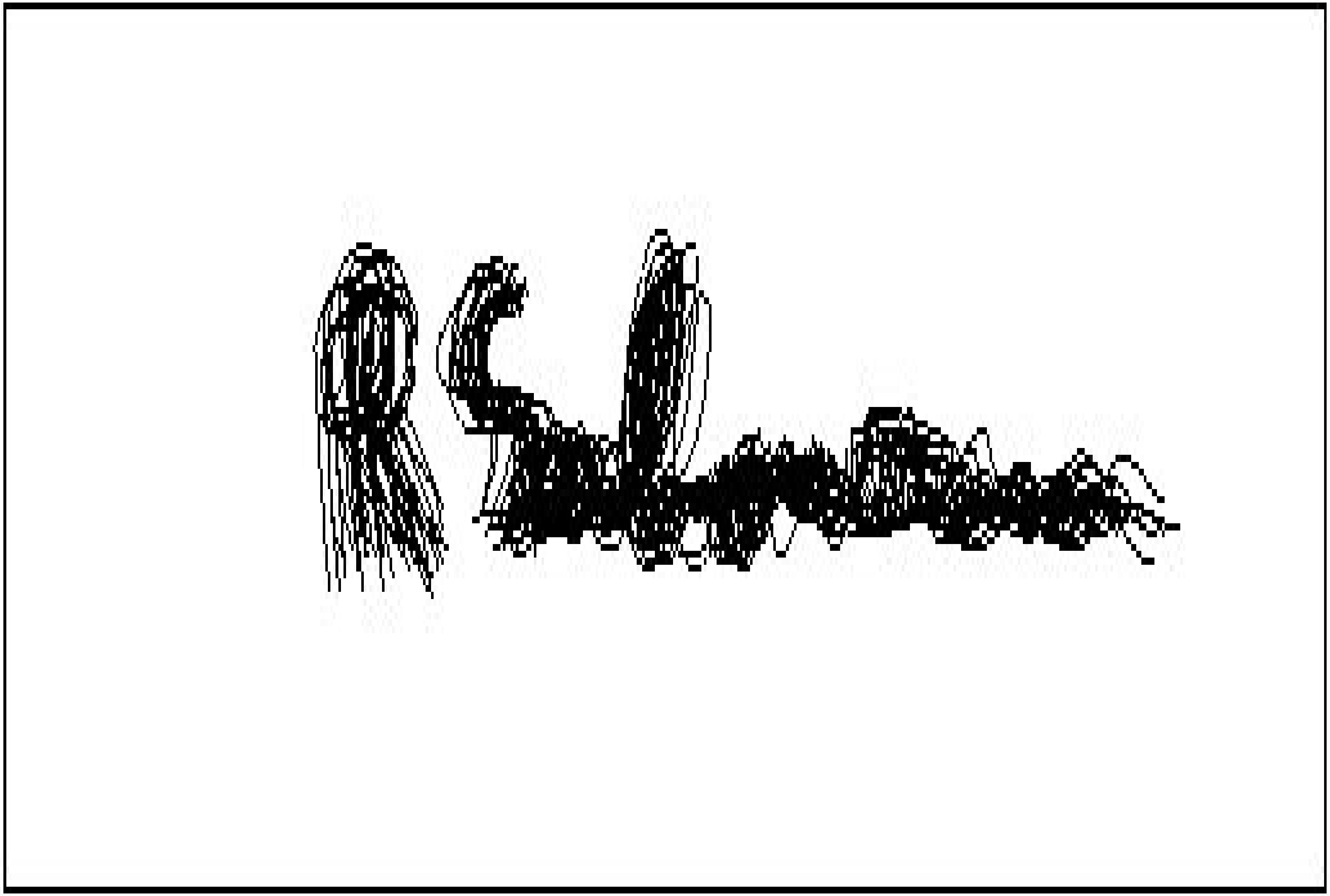


Finger print



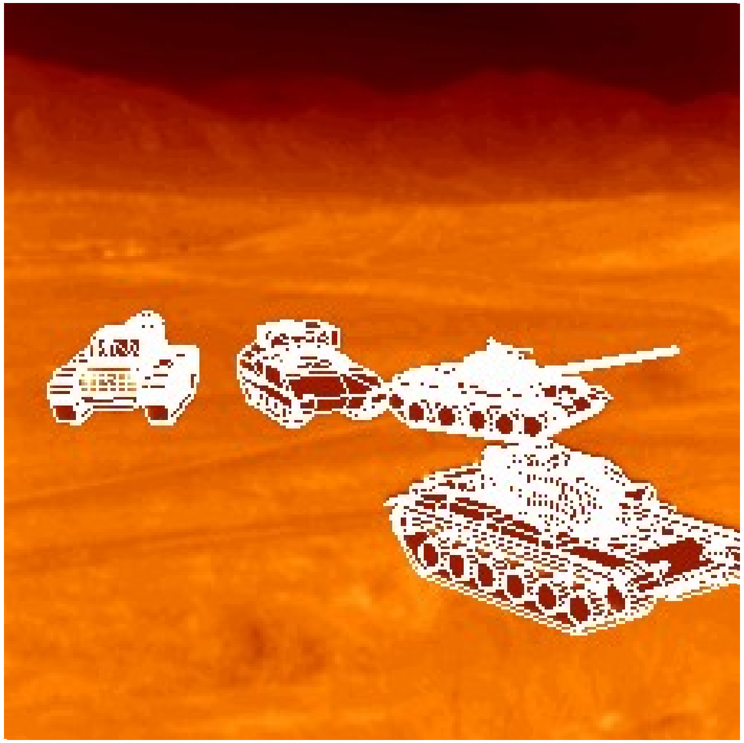
fingerprint image

Signature Verification



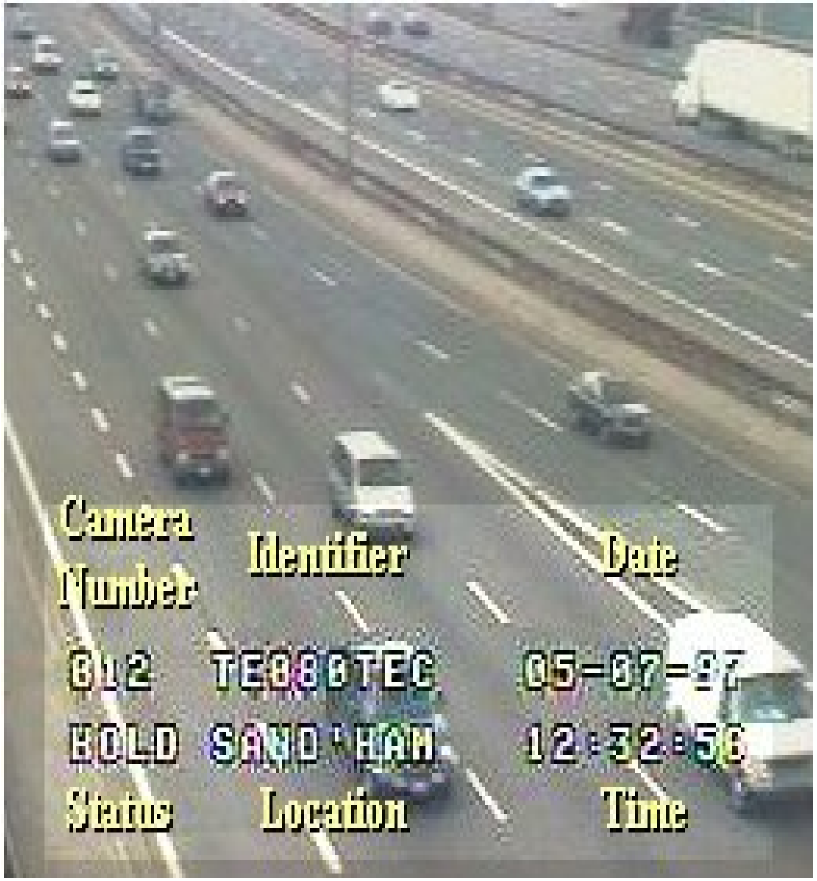
Face Recognition

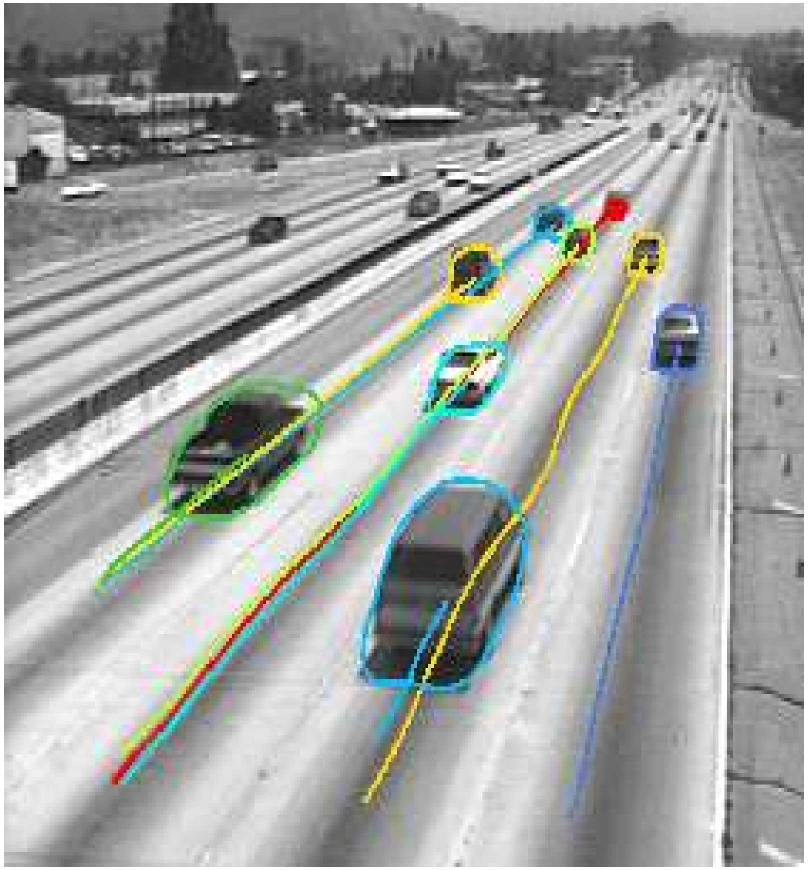
Target Recognition



Robotics vision

Traffic Monitoring





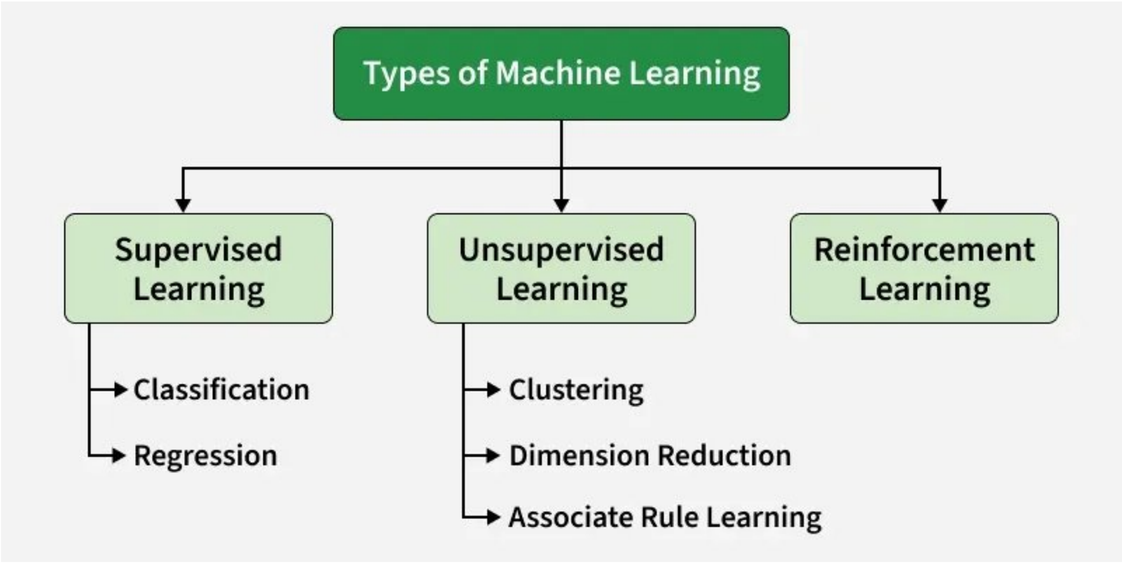
Types of Machine Learning

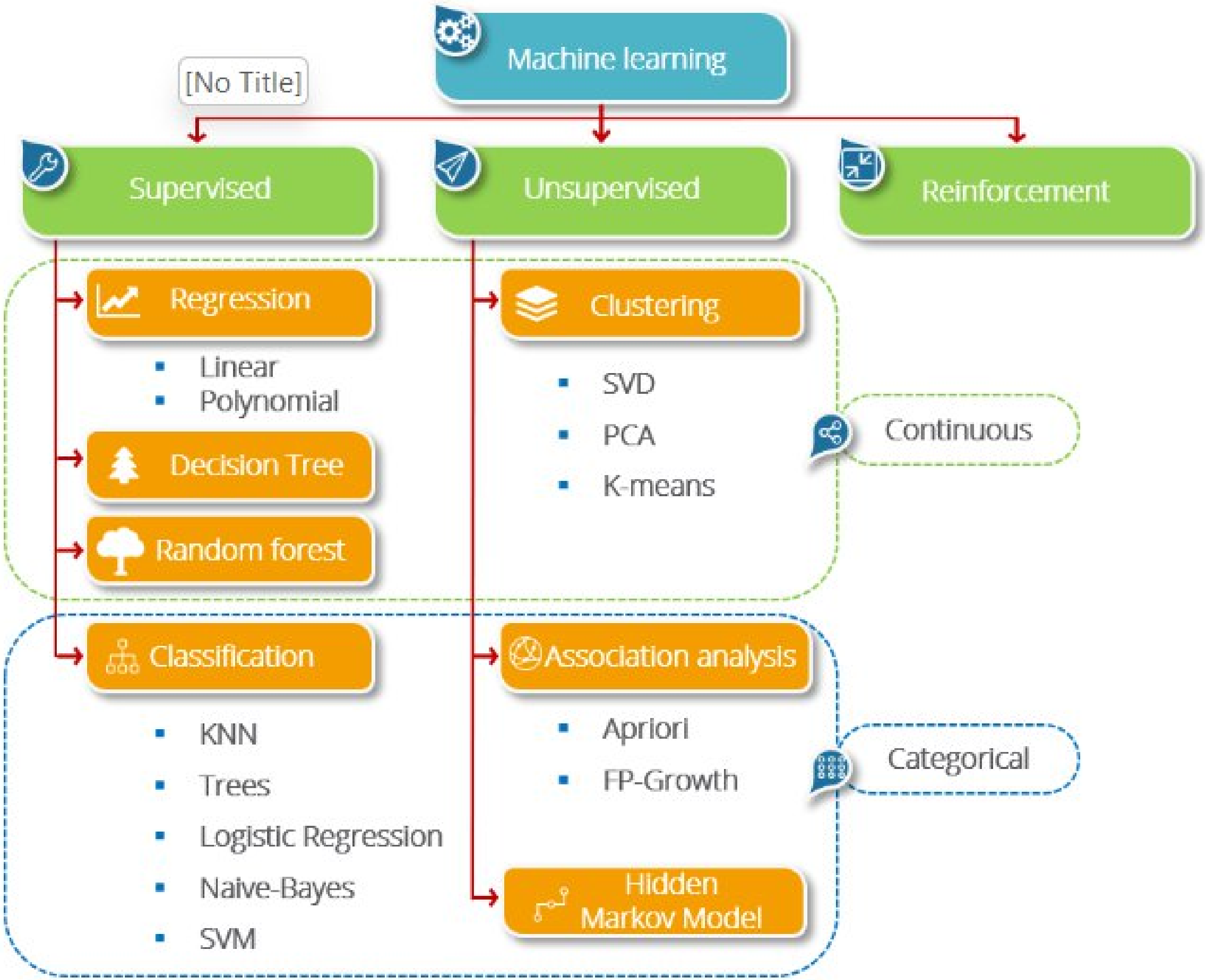
1. **Supervised** learning
   A series of examples are used for feedback

1. **Unsupervised** learning
   No feedback

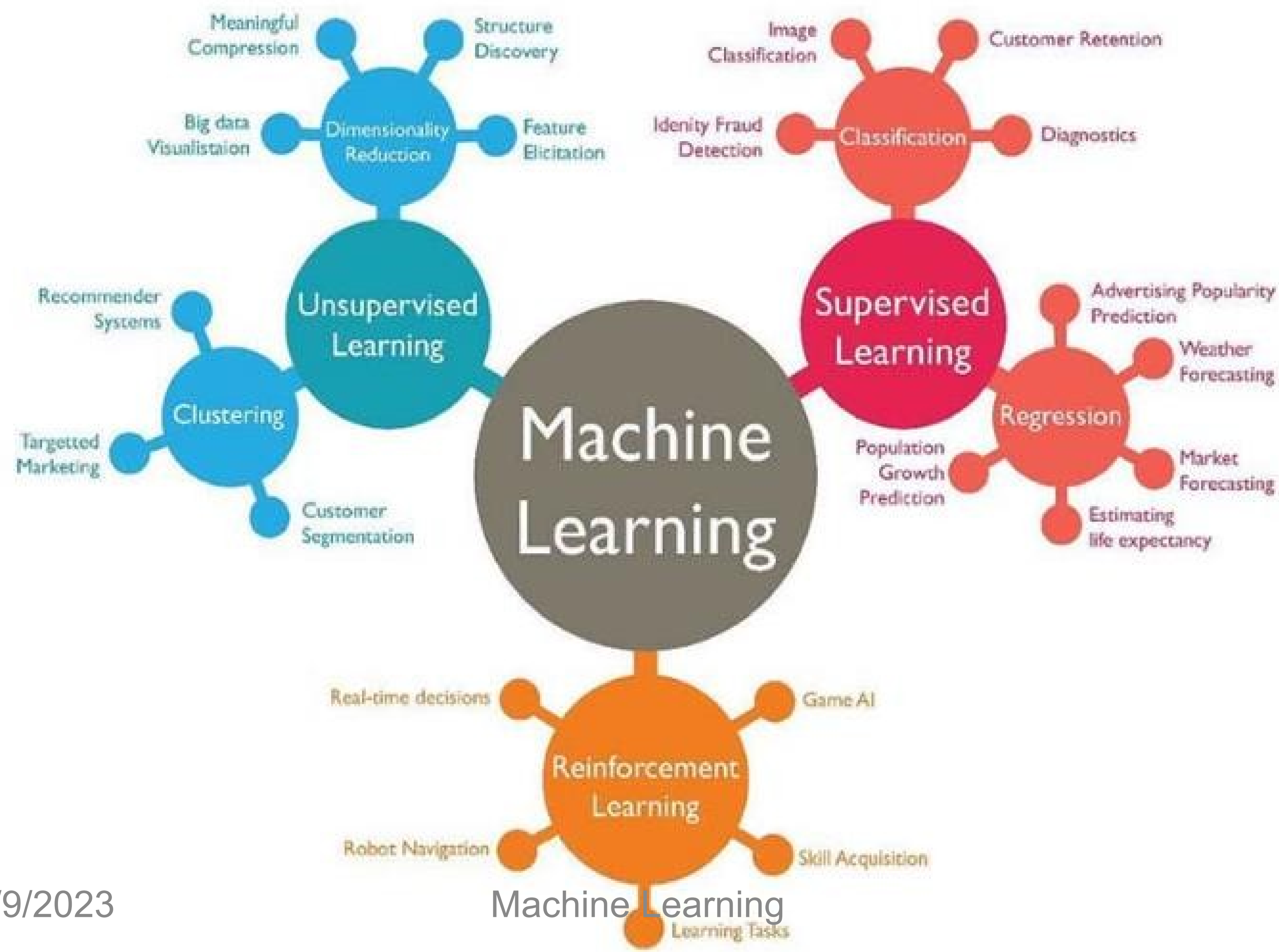
1. **Reinforcement** learning
   Indirect feedback after experience

Additionally, there is a more specific category called Semi-Supervised Learning and Self-Supervised Learning, which combines elements of both supervised and unsupervised learning




[No Title]
Machine learning
Supervised
Unsupervised
Reinforcement
Regression
Linear
Polynomial
Decision Tree
Random forest
Clustering
SVD
PCA
K-means
Continuous
Classification
KNN
Trees
Logistic Regression
Naive-Bayes
SVM
Association analysis
Apriori
FP-Growth
Hidden Markov Model
Categorical

Learning



Machine Learning
Unsupervised Learning
Dimensionality Reduction
Meaningful Compression
Structure Discovery
Big data Visualistaion
Feature Elicitation
Clustering
Recommender Systems
Targetted Marketing
Customer Segmentation
Supervised Learning
Classification
Image Classification
Customer Retention
Idenity Fraud Detection
Diagnostics
Regression
Advertising Popularity Prediction
Weather Forecasting
Population Growth Prediction
Market Forecasting
Estimating life expectancy
Reinforcement Learning
Real-time decisions
Game AI
Robot Navigation
Skill Acquisition
Learning Tasks

Supervised Learning

- A learning task involves a set of input variables and a set of output variables.
- The set of possible relationships (hypotheses) between input and output variables is known as the **hypothesis space**.
- Hypothesis have representations such as numerical functions, symbolic rules, decision trees and artificial neural nets.
- The learning algorithm attempts to find the best hypothesis that maps input to output using *"feedback"*.
- **Feedback** consists of a set of points (training data) for which values of input and output variables are known.

Unsupervised Learning

- In **unsupervised learning**, output variables are not known.
- Unsupervised learning algorithms identify trends in data and make inferences without knowledge of correct answers.

Reinforcement Learning

- It is a feedback-based machine learning technique, whereby an agent learns to behave in an environment by observing his mistakes and performing the actions.
- Reinforcement learning applies the method of learning via Interaction and feedback
- Reinforcement learning trains an agent to make a sequence of decisions through trial and error. The agent interacts with the environment, receives feedback in the form of rewards or penalties and learns optimal actions over time.

What is Supervised Learning?

CAR

CAR

+

=

Training Dataset

BIKE

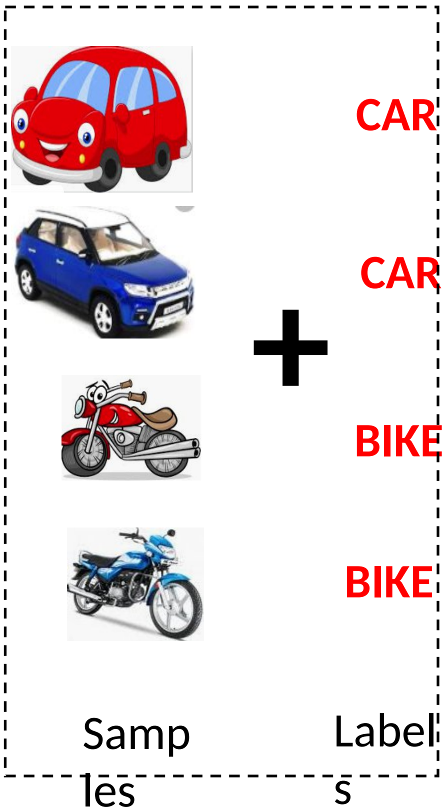
BIKE

Samp
les

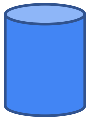
Label
s

In supervised learning, we need a Labelled Training Dataset

What is Supervised Learning?



+ = **Training Dataset**

$f($  $,\quad) =$

Given a labelled dataset, the task is to devise a function which takes the dataset, and a new sample, and produces an output value.

What is Supervised Learning?

CAR

CAR

BIKE

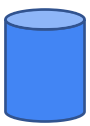
BIKE

Samp
les

Label
s

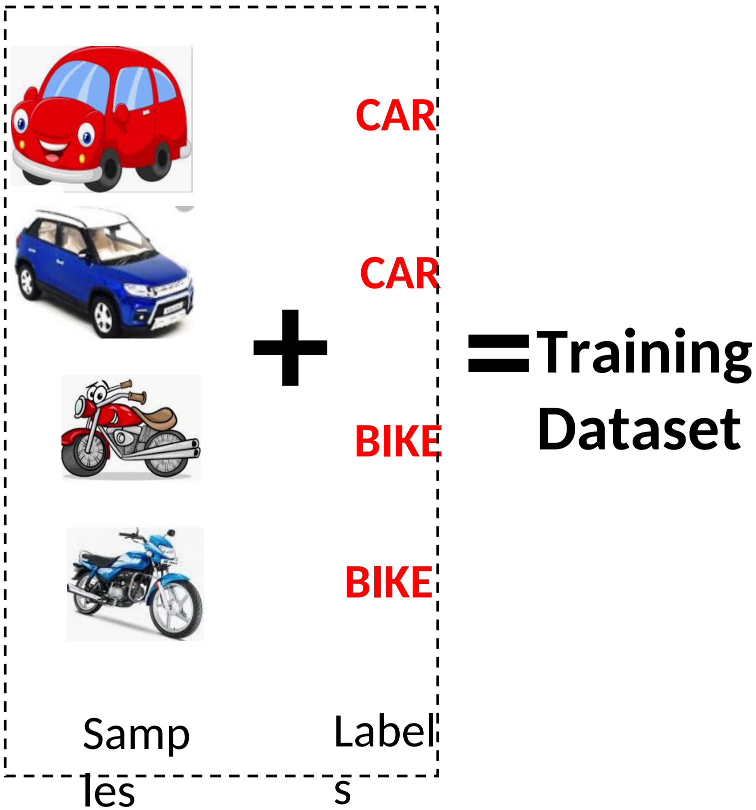
+

= Training Dataset

$f($  ,  $)=$

Given a labelled dataset, the task is to devise a function which takes the dataset, and a new sample, and produces an output value.

What is Supervised Learning?



[Given a labelled dataset, the task is to devise a function which takes the dataset, and a new sample, and produces an output value.]

Classification

CAR

CAR

BIKE

BIKE

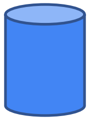
Samp
les

Label
s

+

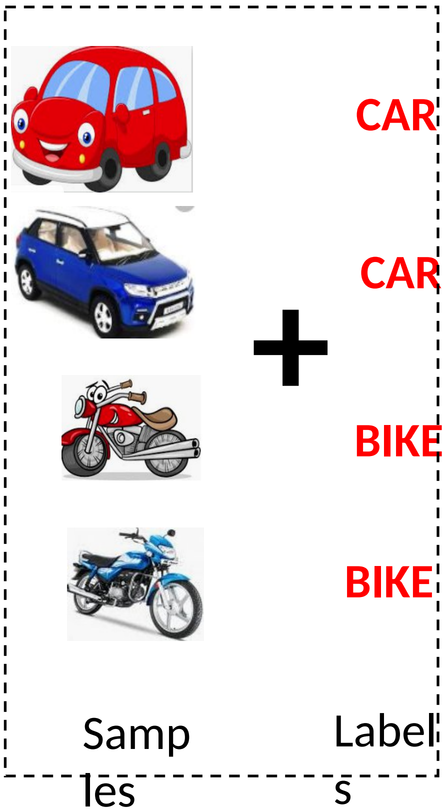
= Training Dataset

Classification

$f($  ,  $) =$ CAR

If the possible output values of the function are predefined and discrete/categorical, it is called Classification

Supervised Learning



+ = Training Dataset

Classification

Predefined classes means, it will produce output only from the labels defined in the dataset. For example, even if we input a bus, it will produce either CAR or BIKE

Classifier

Elepha
nt

Tiger

Dataset

Classifie
r

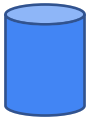
Elepha
nt

Identify the
Animal ?

Regression

Dataset

Regression

$f($  ,  $) = 20500.50$

If the possible output values of the function are continuous real values, then it is called Regression

The classification and Regression problems are supervised, because the decision depends on the characteristics of the ground truth labels or values present in the dataset, which we define as experience

Supervised Learning- Summary

Machines are trained using labeled data to predict results

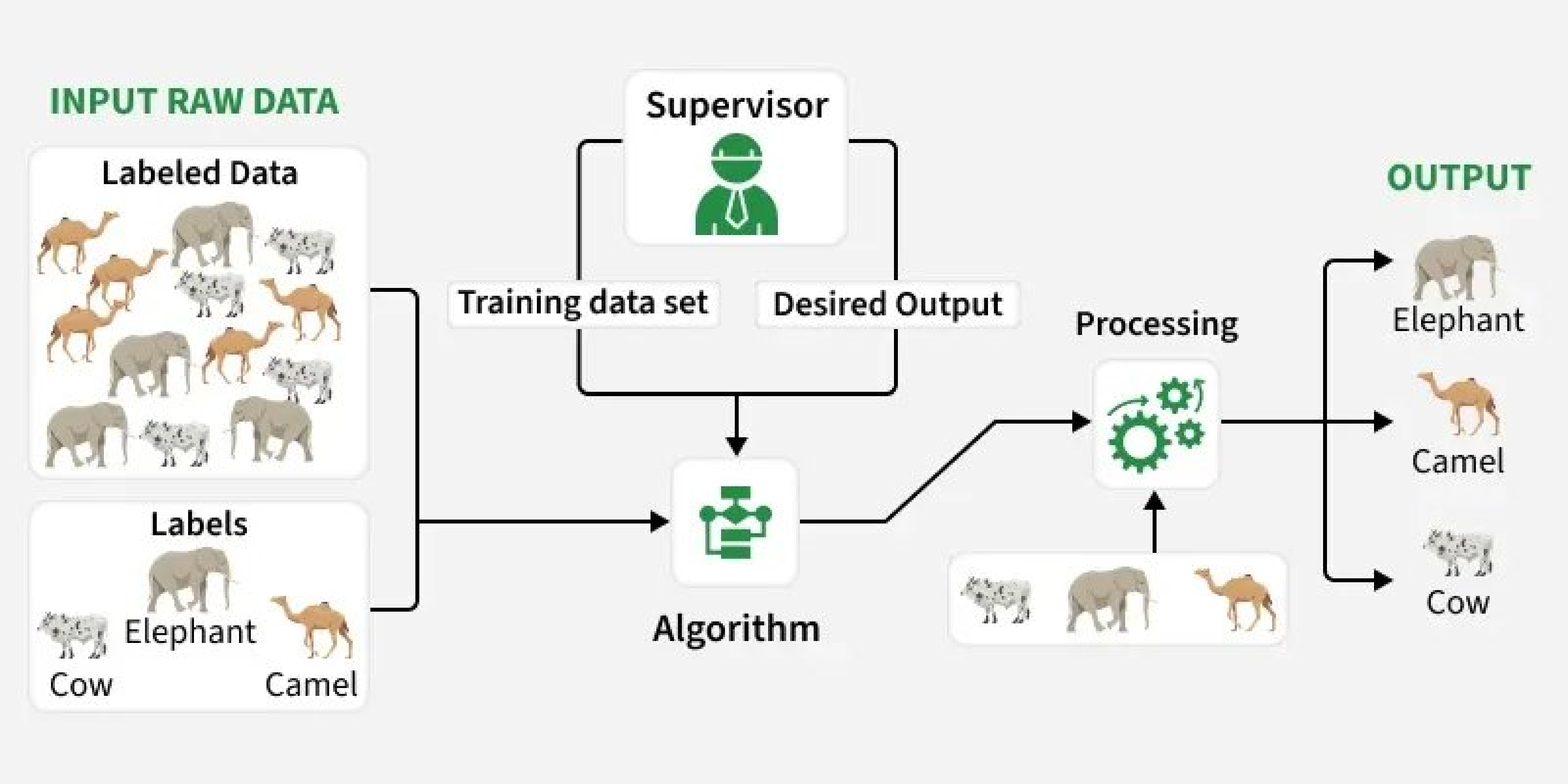
Labeled data → group of data tagged with one or more names, and the data is already known to machines.

- The algorithm measures its accuracy through the loss function, adjusting until the error has been sufficiently minimized.

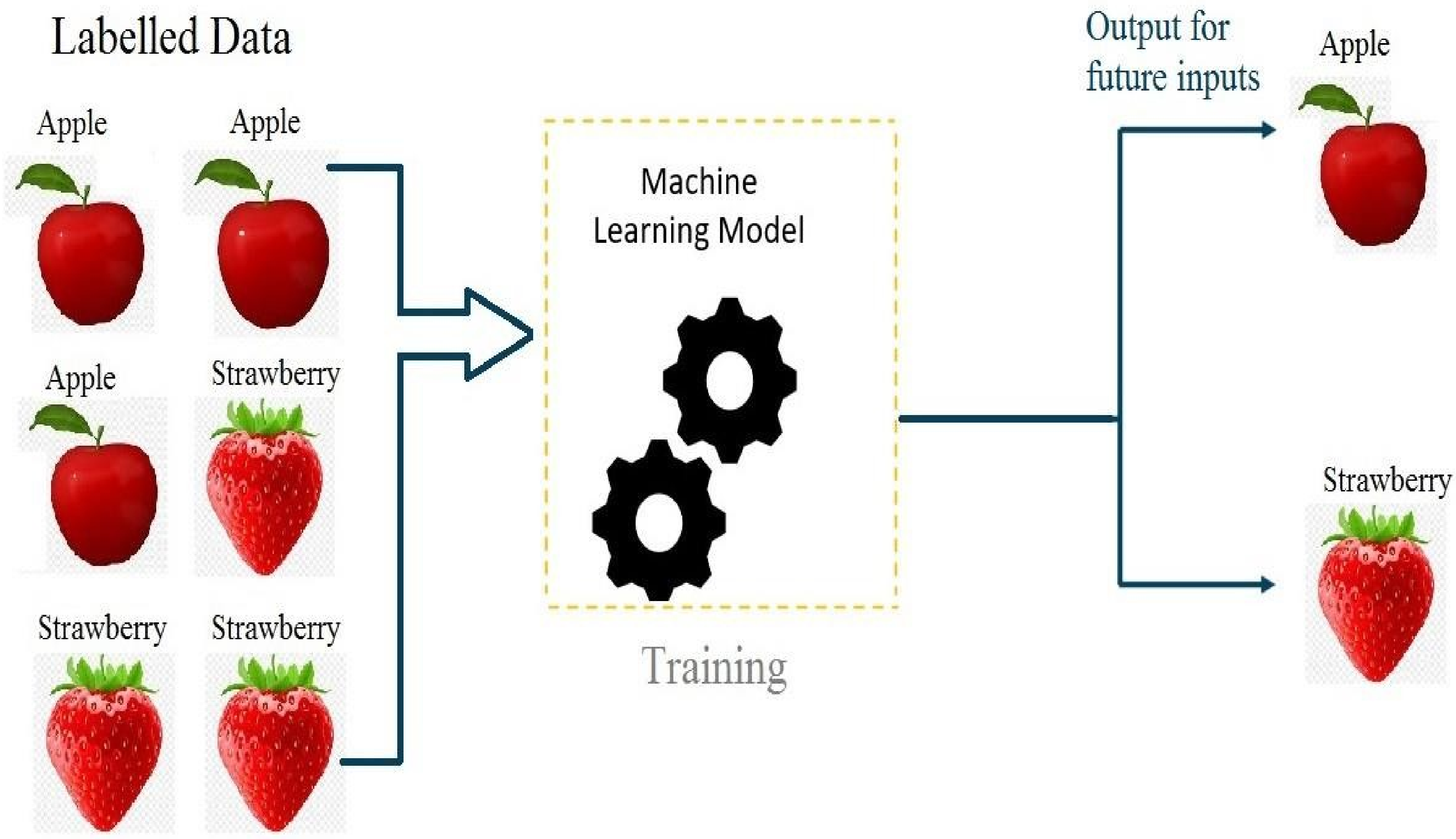
This is similar to a teacher-student scenario.

- There is a teacher who guides the student to learn from books and other materials.
- The student is then tested and if correct, the student passes.
- Else, the teacher tunes the student and makes the student learn from the mistakes that he or she had made in the past.

In Supervised Learning algorithms learn to map points between inputs and correct outputs. It has both training and validation datasets labelled.



Supervised learning



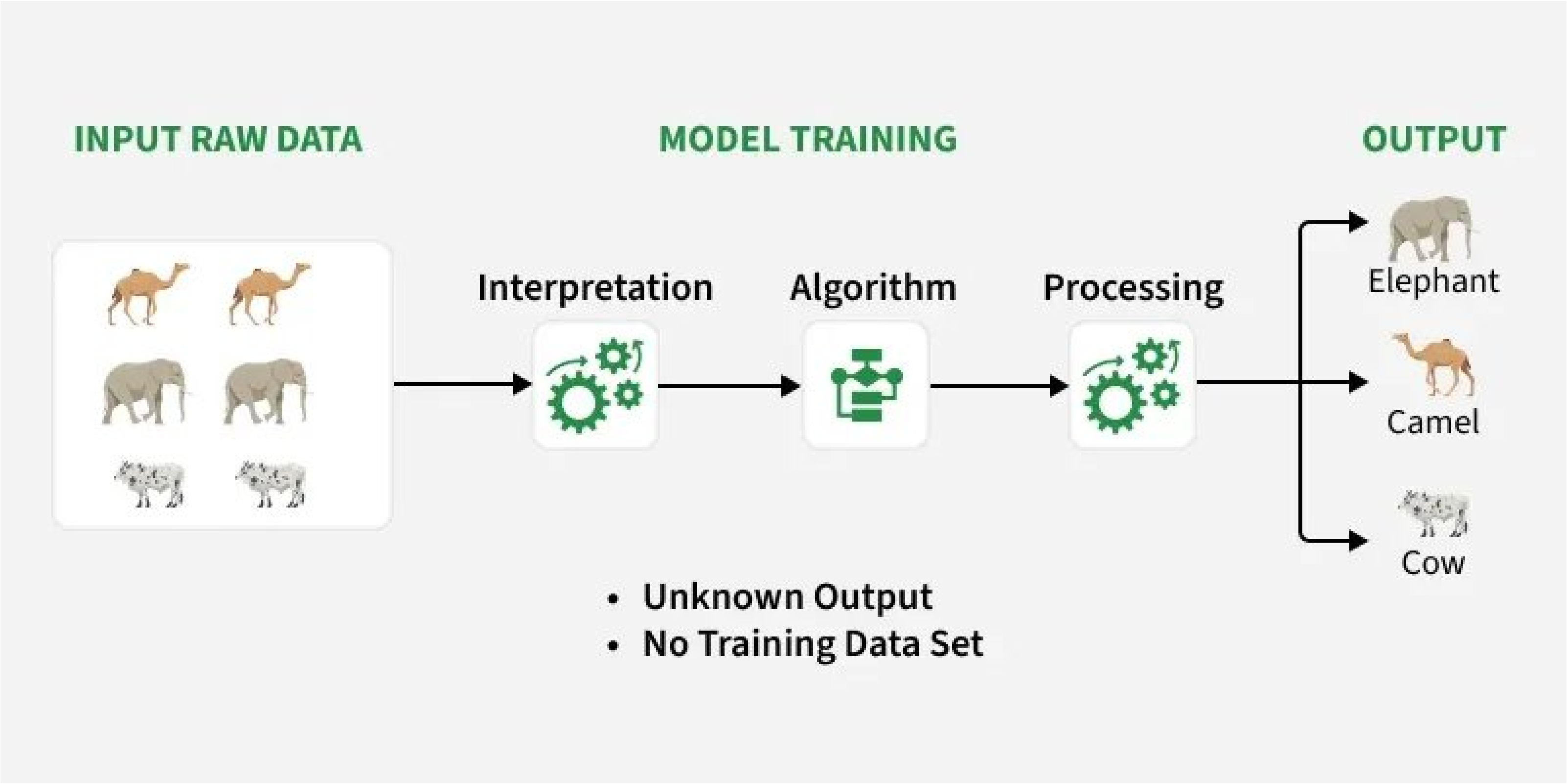
Unsupervised Learning

- Unsupervised Learning works with unlabeled data, meaning there are no predefined outputs. The algorithm finds hidden patterns, groups or relationships within the data on its own.
- Eg : A student who has textbooks and all the required material to study but has no teacher to guide. Ultimately, the student will have to learn by himself or herself to pass the exams.

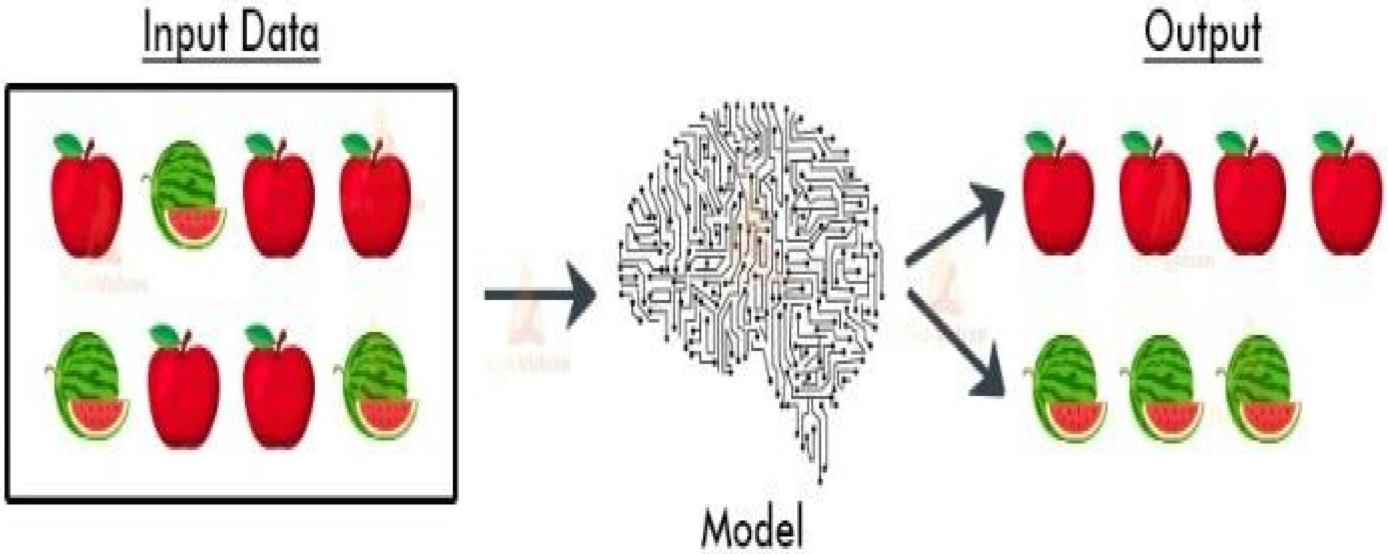
Classified into

- Clustering
- Association

Unsupervised Learning



Unsupervised Learning



What is Unsupervised Learning

~~CAR~~

~~CAR~~

~~BIKE~~

~~BIKE~~

Dataset

In the unsupervised learning, we do not need to know the labels or Ground truth values

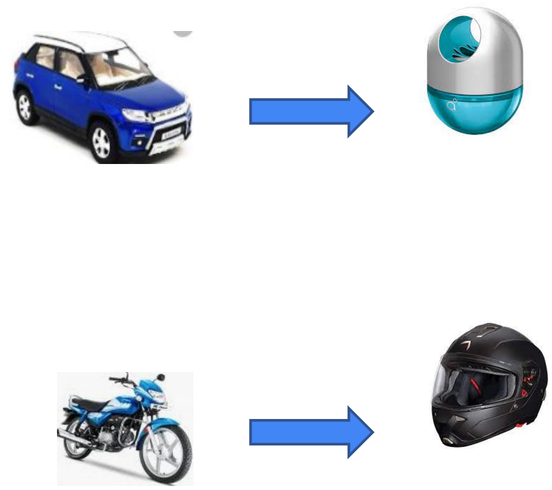
Clustering

Clustering

Dataset

The task is to identify the patterns like group the similar objects together

Association Rules Mining



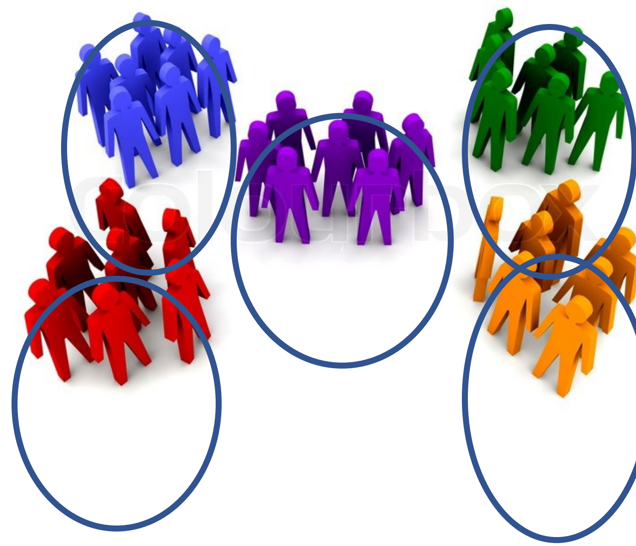
Association Rules Mining

Dataset

[Association rules like]

More Example Unsupervised Learning- Clustering

Dataset



More Example Unsupervised Learning-Association

Applications of unsupervised learning

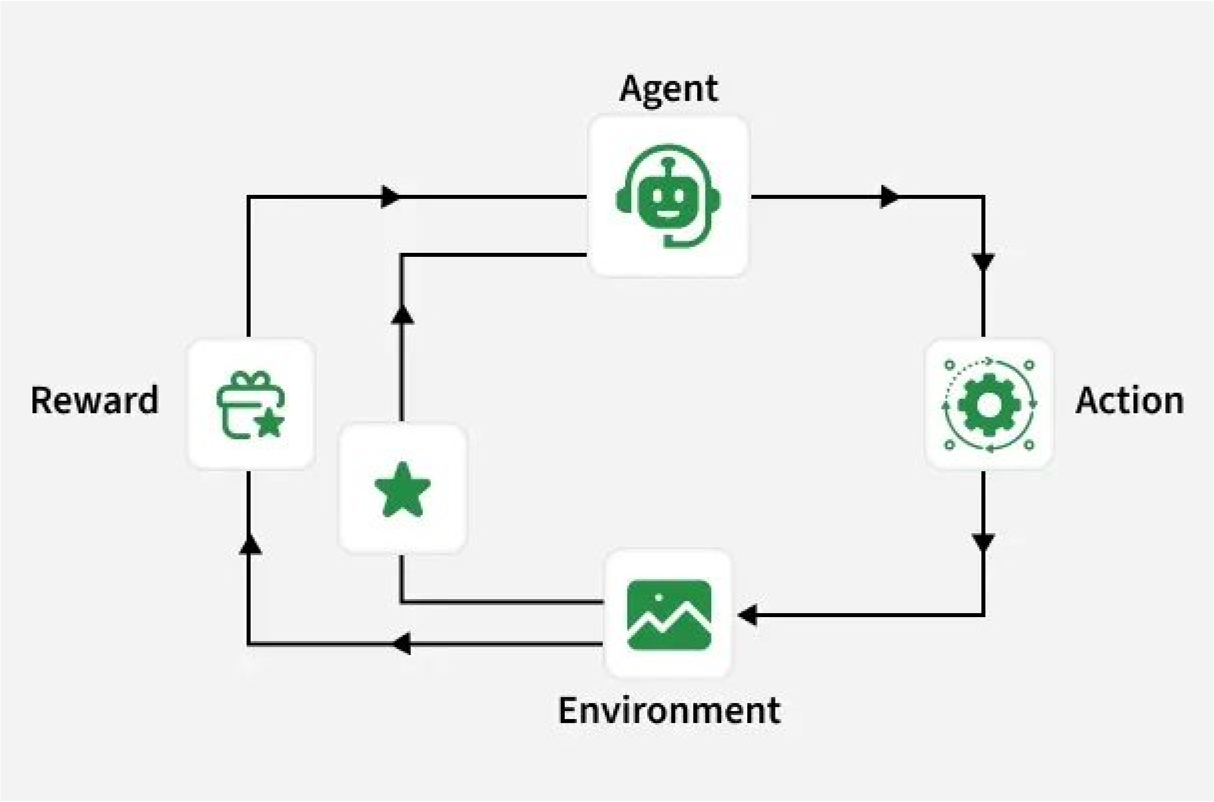
- News Sections
- Computer vision
- Medical imaging
- Anomaly detection
- Recommendation Engines

Reinforcement Learning

- It is a feedback-based machine learning technique, whereby an agent learns to behave in an environment by observing his mistakes and performing the actions.
- Reinforcement learning applies the method of learning via Interaction and feedback
- Reinforcement learning trains an agent to make a sequence of decisions through trial and error. The agent interacts with the environment, receives feedback in the form of rewards or penalties and learns optimal actions over time.

Reinforcement Learning

Learning from trials and errors



Reinforcement Learning

Agent: It is the learner or the decision-maker performing actions to receive a reward.

Environment: It is the scenario where an agent learns and performs future tasks.

Action: actions that are performed by the agent.

State: current situation

Policy: Decision-making function of an agent whereby the agent decides the future action based on the current state.

Reward: Returns provided by the environment to an agent for performing each action.

Example: An AI agent learning to play chess gets positive feedback for good moves and negative for poor ones. Over time, it learns strategies to win more often.

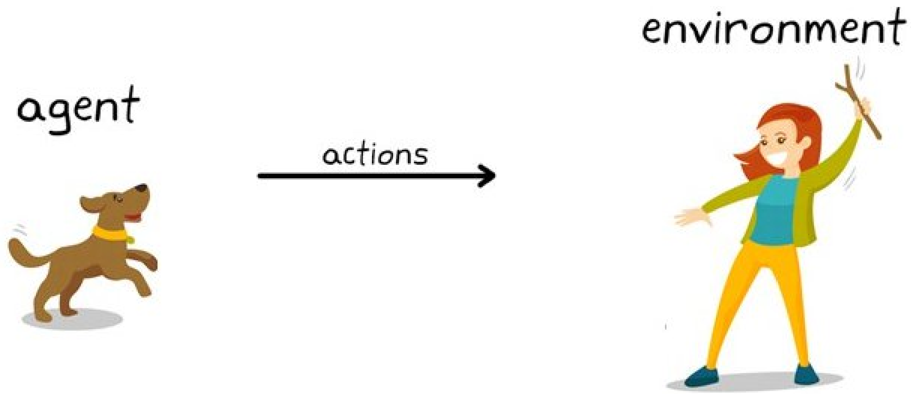
- Agent get a reward for each right action and a penalty for each wrong action.
- Agent learns automatically with feedback and improve its performance.

Here are some of most common reinforcement learning algorithms:

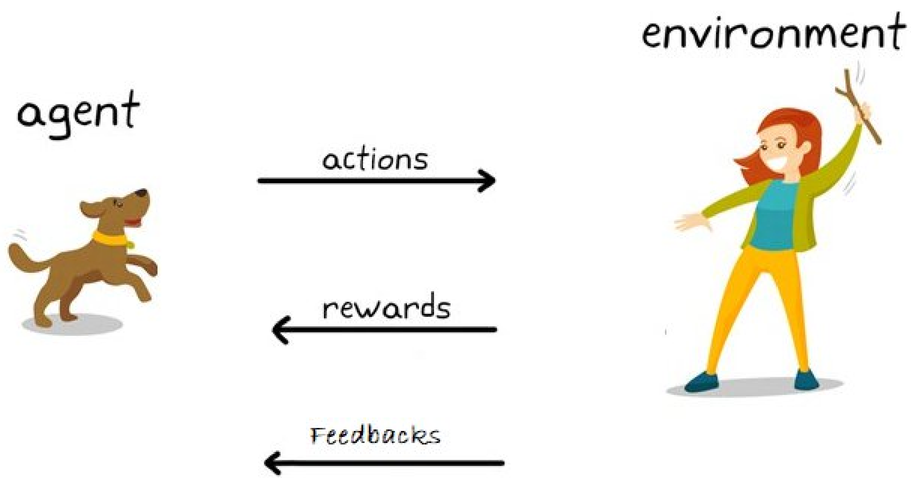
- Q-learning
- SARSA (State-Action-Reward-State-Action)
- Deep Q-learning

What is Reinforcement Learning

What is Reinforcement Learning



What is Reinforcement Learning



Example

Agent

Task

Environme
nt

Reinforcement Learning

**Punishme
nt**

Reinforcement Learning

Reward

Baby Learn from the Trials and Errors

How Does Reinforcement Learning Work- Example

- Cats don’t understand any form of language and therefore a different strategy has to be followed to communicate with the cat.
- A situation is created where the cat acts in various ways. The cat is rewarded with fish if it is the desired way. Therefore the cat behaves in the same way whenever it faces that situation expecting more food as a reward.
- The scenario defines the process of learning from positive experiences.
- Lastly, the cat also learns what not to do through negative experiences.

Where to Use Reinforcement Learning

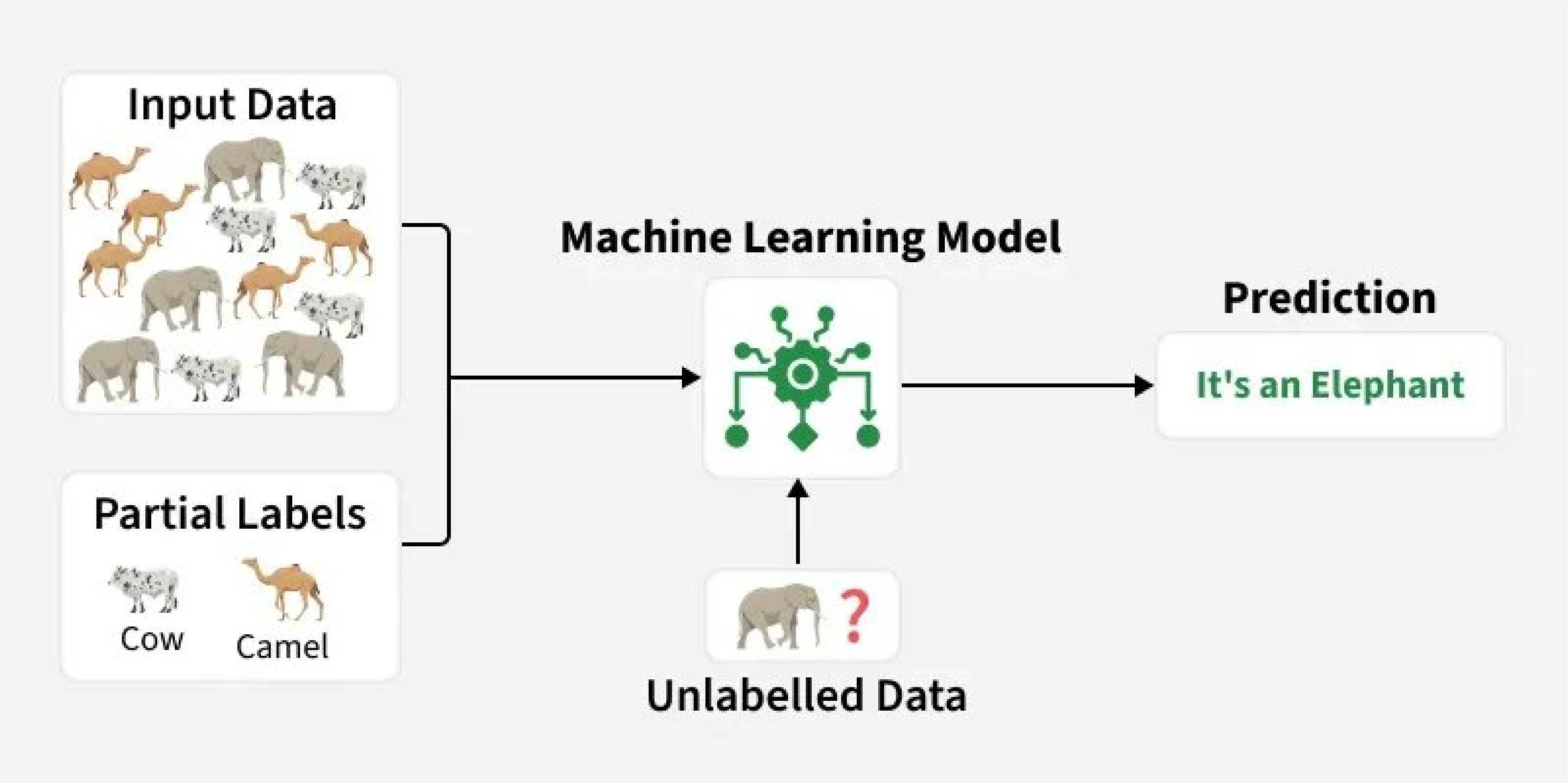
- When you need an agent to learn by interacting with an environment.
- Best for decision-making or optimization tasks involving trial and feedback loops.
- Used when long-term performance or adaptive behavior is more important than immediate accuracy.

Applications of Reinforcement Learning

- **Gaming and simulation:** Teaching agents or NPCs to play and adapt intelligently.
- **Robotics and automation:** Enabling robots to perform tasks autonomously.
- **Autonomous vehicles:** Helping self-driving cars make real-time decisions.
- **Healthcare and finance:** Optimizing treatment plans, trading and resource allocation.
- **Recommendation and personalization:** Improving user experience through adaptive suggestions.
- **Industrial and energy management:** Optimizing control systems and energy use.

Semi-Supervised Learning: Supervised + Unsupervised Learning

- Semi-Supervised Learning combines both Supervised and Unsupervised approaches.
- It uses a small set of labeled data and a large set of unlabeled data for training
- Useful when labeling is costly or time-consuming.



Semi-Supervised Learning: Supervised + Unsupervised Learning

Example: Consider that we are building a language translation model, having labeled translations for every sentence pair can be resources intensive.

It allows the models to learn from labeled and unlabeled sentence pairs, making them more accurate.

This technique has led to significant improvements in the quality of machine translation services.

Popular Techniques

- **Graph-based Learning:** Spreads label information through data relationships.
- **Label Propagation:** Iteratively assigns labels to unlabeled data.
- **Co-training**: Uses two models to train and label each other's data.
- **Self-training**: Uses model predictions as pseudo-labels.
- **Generative Adversarial Networks (GANs)**: Generates synthetic data to improve learning.

Where to Use Semi-Supervised Learning

- When you have limited labeled data but plenty of unlabeled data.
- Useful for domains with high labeling costs, such as medical, NLP or image datasets.
- Ideal when unlabeled data still holds valuable information that can improve learning performance.

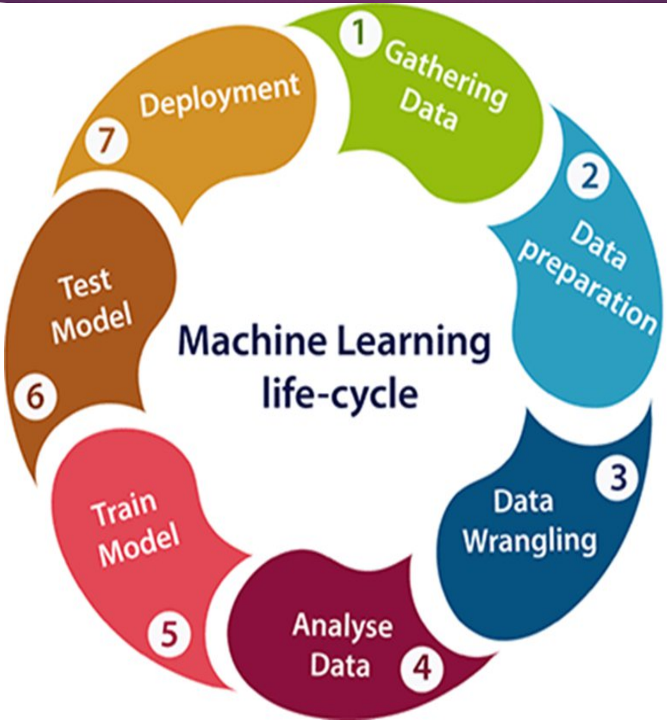
Applications

- **Image Classification:** Combine small labeled and large unlabeled image datasets to improve accuracy.
- **Natural Language Processing (NLP):** Enhance language models by using a mix of labeled and vast unlabeled text data.
- **Speech Recognition:** Boost accuracy by leveraging limited transcribed audio and more unlabeled speech data.
- **Recommendation Systems:** Improve recommendations using sparse labeled data and abundant unlabeled user behavior.
- **Healthcare & Medical Imaging:** Improve medical image analysis with a mix of labeled and

Comparison

Type	Data Requirement	Label Availability	Learning Goal	Common Use Case
Supervised	High	Labeled	Predict outputs	Spam detection, Price prediction
Unsupervised	Medium	Unlabeled	Find hidden patterns	Customer segmentation
Reinforcement	High	Reward feedback	Learn best actions	Robotics, Games

Machine learning Life cycle



Gathering Data

- Data Gathering is the first step of the machine learning life cycle. The goal of this step is to identify and obtain all data-related problems.
- Need to identify the different data sources, as data can be collected from various sources such as files, database, internet, or mobile devices.
- The quantity and quality of the collected data will determine the efficiency of the output.
- The more will be the data, the more accurate will be the prediction.

Gathering Data(cont.)

- This step includes the below tasks:
  - **Identify various data sources**
  - **Collect data**
  - **Integrate the data obtained from different sources**
- By performing the above task, we get a coherent set of data, also called as a **dataset**. It will be used in further steps

Data preparation

- After collecting the data, prepare it for further steps. Data preparation is a step where we put our data into a suitable place and prepare it to use in our machine learning training.
- Put all data together, and then randomize the ordering of data.

Further divided into two processes:

- **Data exploration:**
  Nature of data that we have to work with. We need to understand the characteristics, format, and quality of data.
  A better understanding of data leads to an effective outcome. In this, we find Correlations, general trends, and outliers.
- **Data pre-processing:**
  Now the next step is pre-processing of data for its analysis.

Data Wrangling

- Process of cleaning and converting raw data into a useable format.
- It is not necessary that data we have collected is always of our use as some of the data may not be useful. In real-world applications, collected data may have various issues, including:
  - Missing Values
  - Duplicate data
  - Invalid data
  - Noise
- Use various filtering techniques to clean the data.
- It is mandatory to detect and remove the above issues because it can negatively affect the quality of the outcome

Data Analysis

- Cleaned and prepared data is passed on to the analysis step. This step involves:
- Selection of analytical techniques
- Building models
- Review the result
- The aim is to build a machine learning model to analyze the data using various analytical techniques and review the outcome.
- Use Classification, Regression, Cluster analysis, Association, etc. then build the model using prepared data, and evaluate the model.

Train Model

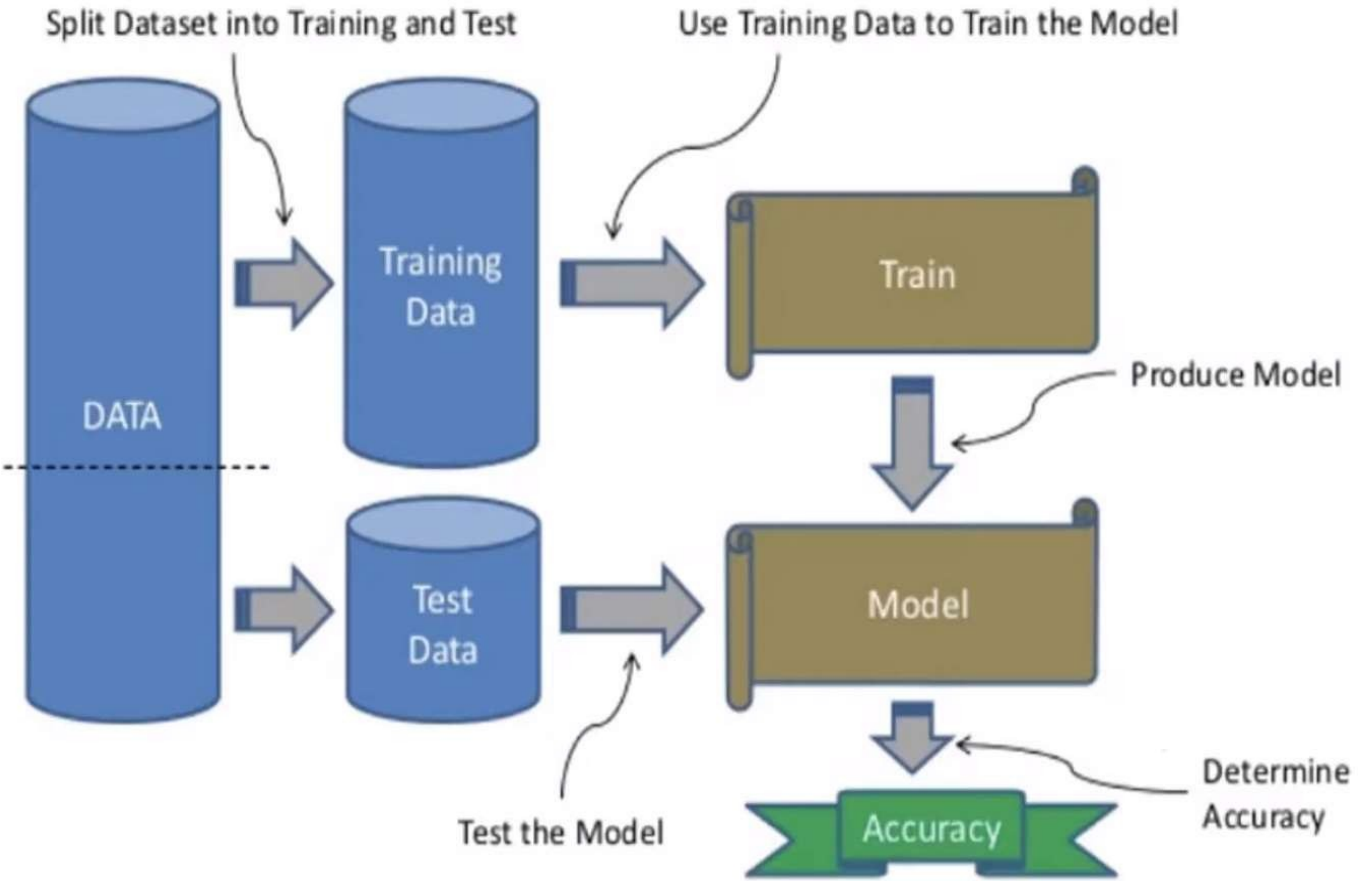
- Train the model to improve its performance for better outcome of the problem.

- Use datasets to train the model using various machine learning algorithms. Training a model is required so that it can understand the various patterns, rules, and, features.

Test Model

- Once machine learning model has been trained on a given dataset, then test the model. In this step, check for the accuracy of the model by providing a test dataset to it.

- Testing the model determines the percentage accuracy of the model as per the requirement of project or problem.


Split Dataset into Training and Test
Use Training Data to Train the Model
DATA
Training Data
Test Data
Train
Produce Model
Model
Test the Model
Determine Accuracy
Accuracy

Deployment

- The last step of machine learning life cycle is deployment, where we **deploy the model in the real-world system.**
- If the above-prepared model is producing an accurate result as per our requirement with acceptable speed, then we deploy the model in the real system.
- Similar to making the final report for a project.

Implementation for model Training

1. Import Libraries

```
import numpy as np

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler, OneHotEncoder

from sklearn.pipeline import Pipeline

from sklearn.compose import ColumnTransformer

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score
```

2. Load and Prepare the data

```
# Load dataset
df =
pd.read_csv("https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.cs
v")


# Select relevant features
features = ['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked']
df = df[features + ['Survived']].dropna()  # Drop rows with missing values


# Display the first few rows
print(df.head())
```

```
   Pclass     Sex   Age  SibSp  Parch     Fare Embarked  Survived
0       3    male  22.0      1      0   7.2500        S         0
1       1  female  38.0      1      0  71.2833        C         1
2       3  female  26.0      0      0   7.9250        S         1
3       1  female  35.0      1      0  53.1000        S         1
4       3    male  35.0      0      0   8.0500        S         0
```

3. Define Preprocessing Steps

```
# Define numerical and categorical features
num_features = ['Age', 'SibSp', 'Parch', 'Fare']
cat_features = ['Pclass', 'Sex', 'Embarked']

# Define transformers
num_transformer = StandardScaler()  # Standardization for numerical features
cat_transformer = OneHotEncoder(handle_unknown='ignore')  # One-hot encoding for categorical features

# Combine transformers into a preprocessor
preprocessor = ColumnTransformer([('num', num_transformer, num_features),
    ('cat', cat_transformer, cat_features)])
```

4. Split the data for training and Testing

```
# Define target and features
X = df[features]
y = df['Survived']
# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Display the shape of the data
print(f"Training set shape: {X_train.shape}")
print(f"Testing set shape: {X_test.shape}")
```

Output:

```
Training set shape: (567, 7)
Testing set shape: (143, 7)
```

5. Build and Train model

```
# Define the pipeline
pipeline = Pipeline([
    ('preprocessor', preprocessor),  # Data transformation
    ('classifier', RandomForestClassifier(n_estimators=100,
random_state=42))  # ML model
])
# Train the model
pipeline.fit(X_train, y_train)
print("Model training complete!")
```

Output:

```
Model training complete!
```

6. Evaluate the Model

```
# Make predictions
y_pred = pipeline.predict(X_test)

# Compute accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy:.2f}")
```

Output:

```
Model Accuracy: 0.76
```

7. Save and Load the Model

```python
import joblib

# Save the trained pipeline

joblib.dump(pipeline, 'ml_pipeline.pkl')

# Load the model

loaded_pipeline = joblib.load('ml_pipeline.pkl')

# Predict using the loaded model

sample_data = pd.DataFrame([{'Pclass': 3, 'Sex': 'male', 'Age': 25, 'SibSp':
0, 'Parch': 0, 'Fare': 7.5, 'Embarked': 'S'}])

prediction = loaded_pipeline.predict(sample_data)

print(f"Prediction: {'Survived' if prediction[0] == 1 else 'Did not Survive'}")
```

Output:

```
Prediction: Did not Survive
```

```
Data Sample:
   Pclass     Sex   Age  SibSp  Parch     Fare Embarked  Survived
0       3    male  22.0      1      0   7.2500        S         0
1       1  female  38.0      1      0  71.2833        C         1
2       3  female  26.0      0      0   7.9250        S         1
3       1  female  35.0      1      0  53.1000        S         1
4       3    male  35.0      0      0   8.0500        S         0
Training set shape: (569, 7)
Testing set shape: (143, 7)
Model training complete!
Model Accuracy: 0.76
Prediction for Sample Data: Did not Survive
```

Learning

“The action or process of obtaining information or ability through studying, practicing, being instructed, or experiencing something,”

We may split learning techniques into five categories based on this:

1. **Rote Learning (Memorizing):** Memorizing things without understanding the underlying principles or rationale.
2. **Instructions (Passive Learning):** Learning from a teacher or expert.
3. **Analogy (Experience)**: We may learn new things by applying what we’ve learned in the past.
4. **Inductive Learning (Experience):** Formulating a generalized notion based on prior experience.
5. **Deductive Learning:** Getting new information from old information.

Inductive Learning

- Inductive Learning is the process of forming a generalized notion after viewing many examples of the concept.
- For instance, suppose a child is requested to write the solution 2*8=? They can memorize the answer using the rote learning approach or use inductive Learning to construct a notion to compute the outcomes using examples such as 2*1=2, 2*2=4, and so on. In this approach, the child will be able to answer questions of a similar nature utilizing the same notion.
- Similarly, we may train our computer to learn from previous data and recognize whether an object is in a given category of interest.

The hypothesis of Inductive Learning:

- The ultimate aim of concept learning is to find a hypothesis ‘h’ that is identical to the target notion c over data set X, with the only knowledge about c being its value over X.
- “Any hypothesis that approximates the target value well over a suitably large collection of training instances will likewise approximate the target value well over other unseen cases,”
- Consider whether a person goes to the movies or not based on four binary characteristics with two potential values (true or false):
  - 1. Is Rich -> true, false
  - 2. Is There Any Free Time -> true or false
  - 3. It’s a Holiday -> true or false
  - 4. Has work pending -> true, false
- We also have training data, with two data items serving as positive samples and one serving as a negative sample:
  - **x1**: <true, true, false, false> : **+ve**
  - **x2**: <true, false, false, true> : **+ve**
  - **x3:** <true, false, false, true> : **-ve**

Concept Learning

What if machines could learn the same way we do, by forming concepts from examples?

- In machine learning, the process of identifying a general rule or concept from specific examples is called concept learning.
- “Inferring a boolean-valued function from training examples of its input and output’
- Formal Definition: It is the process of inferring a generalised description (concept) from a set of positive and negative examples using logical expressions or hypothesis space.

Concept Learning Example:

Let’s say you’re building a model to identify “fruit”.

You provide examples like:

Colour	Shape	Taste	Fruit?
Red	Round	Sweet	Yes
Yellow	Long	Sweet	Yes
Green	Spiky	Sour	No
Orange	Round	Sweet	Yes

From these, the model tries to find a pattern, maybe “sweet and round” means fruit. This is a basic concept learning task in machine learning. The machine tries to find the common features in all “Yes” examples while rejecting the “No” ones.

The Concept Learning Task

The actual concept learning task involves:

1. **Hypothesis Space (H):** All possible concepts the model could learn.
2. **Training Examples (D):** Positive and negative data samples.
3. **Target Concept (C):** The actual rule that classifies inputs correctly.
4. **Learning Algorithm:** The method to search and narrow down hypotheses.

The Goal: Find the hypothesis $h \in H$ such that $h(x) = C(x)$ for every training example x.

concept learning -"Problem of searching through a predefined space of potential hypotheses for the hypothesis that best fits the training examples"

A Concept Learning Task – Enjoy Sport Training Examples

Example	**Sky**	**AirTemp**	**Humidity**	**Wind**	**Water**	**Forecast**	**EnjoySport**
1	Sunny	Warm	Normal	Strong	Warm	Same	**YES**
2	Sunny	Warm	High	Strong	Warm	Same	**YES**
3	Rainy	Cold	High	Strong	Warm	Change	**NO**
4	Sunny	Warm	High	Strong	Warm	Change	**YES**

ATTRIBUTES

CONCEPT

- A set of example days, and each is described by six attributes.
- The task is to learn to predict the value of EnjoySport for arbitrary day, based on the values of its attribute values.

A Concept Learning Task – Enjoy Sport

- Now the target concept is **EnjoySport : X →{0,1}.**
- With this, a learner task is to learn to predict the value of EnjoySport for arbitrary day, based on the values of its attribute values.
- When a new sample with the values for attributes **<Sky, Air Temperature, Humidity, Wind, Water, Forecast>** is given, the value for EnjoySport (ie. 0 or 1) is predicted based on the previous learning.

A Concept Learning Task – Enjoy Sport

Let H denote the set of all possible hypotheses that the learner may consider regarding the identity of the target concept.

So H = {h1, h2, …. }.

Let each hypothesis be a vector of six constraints, specifying the values of the six attributes **<Sky, Air Temperature, Humidity, Wind, Water, Forecast>**.

In hypothesis representation, value of each attribute could be either

- “?’ — that any value is acceptable for this attribute,
- specify a single required value (e.g., Warm) for the attribute, or
- “0” that no value is acceptable.

A Concept Learning Task – Enjoy Sport

For Example:

- The hypothesis that Sachin enjoys his favorite sport only on cold days with high humidity (independent of the values of the other attributes) is represented by the expression **< ?,cold, High,?, ? ,?>**
- The most general hypothesis — < ?, ? , ? , ?, ? , ?> that every day is a positive example
- The most specific hypothesis — < 0, 0, 0, 0, 0, 0>that no day is a positive example.

EnjoySport concept learning task requires

- learning the sets of days for which EnjoySport=yes and then describing this set by a conjunction of constraints over the instance attributes.

A Concept Learning Task – Enjoy Sport

1. **X —** The set of items over which the concept is defined is called the set of instances, which we denote by X.
2. **c —** The concept or function to be learned is called the target concept. In the current example, the target concept corresponds to the value of the attribute EnjoySport (i.e, c(x)=1 if EnjoySport=Yes, and c(x)=0 if EnjoySport= No).
3. **(x, c(x)) —** When learning the target concept, the learner is presented by a set of training examples, each consisting of an instance x from X, along with its target concept value c(x). Instances for which c(x) = 1 are called positive examples and instances for which c(x) = 0 are called negative examples. We will often write the ordered pair (x, c(x)) to describe the training example consisting of the instance x and its target concept value c(x).
4. **D —** We use the symbol D to denote the set of available training examples.
5. **H —** We use the symbol H to denote the set of all possible hypotheses that the learner may consider regarding the identity of the target concept.
6. **h(x) —** In general, each hypothesis h in H represents a Boolean-valued function defined over X; that is, $h : X \rightarrow \{0, 1\}$. The goal of the learner is to find a hypothesis h such that h(x) = c(x) for all x in X.

A Concept Learning Task – Enjoy Sport

Given

- *Instances X* : set of all possible days, each described by the attributes
  - Sky – (values: Sunny, Cloudy, Rainy)
  - AirTemp – (values: Warm, Cold)
  - Humidity – (values: Normal, High)
  - Wind – (values: Strong, Weak)
  - Water – (values: Warm, Cold)
  - Forecast – (values: Same, Change)
- *Target Concept (Function) c* : EnjoySport : $X \rightarrow \{0,1\}$
- *Hypotheses H* : Each hypothesis is described by a conjunction of constraints on the attributes.
- *Training Examples D* : positive and negative examples of the target function

Determine

- A hypothesis h in H such that h(x) = c(x) for all x in D.

Inductive learning hypothesis

For a given collection of examples, in reality, learning algorithm return a function h (hypothesis) that approximates c (target concept). But the expectation is, the learning algorithm to return a function h (hypothesis) that equals c (target concept) ie. $h(x) = c(x)$ for all x in D.

Inductive learning hypothesis is any hypothesis found to approximate the target function well over a sufficiently large set of training examples will also approximate the target function well over any other unobserved examples.

Hypothesis Space- EnjoySport learning task.

X -denote the instances

H -hypotheses

Attributes	Values	Count
1. Sky	Sunny, Cloudy, Rainy	3
2. AirTemp	Warm, Cold	2
3. Humidity	Normal, High	2
4. Wind	Strong, Weak	2
5. Water	Warm, Cool	2
6. Forecast	Same, Change	2

$$Distinct\ Observations\ in\ X = 3.2.2.2.2.2 = 96$$

$$Distinct\ hypothesis\ in\ H = 5.4.4.4.4.4 = 5120$$

Hypothesis h is a vector of six constraints, specifying the values of the six attributes **<Sky, Air Temperature, Humidity, Wind, Water, Forecast>**.

In this hypothesis representation, value of each attribute could be either “?” or “0” other than defined values.

Hypothesis Space- EnjoySport learning task.

The number of combinations: $5 \times 4 \times 4 \times 4 \times 4 \times 4 = 5120$ syntactically distinct hypotheses. (Two more values for attributes: ? and 0)

They are syntactically distinct but not semantically.

For example, the below 2 hypothesis says the same but they look different.

h1 = <Sky=0 AND Temp=warm AND Humidity=? AND Wind=strong AND Water=warm AND Forecast=same >

h2 = <Sky=sunny AND Temp=warm AND Humidity=? AND Wind=strong AND Water=0 AND Forecast=same>

Neither of these hypotheses accept any “day”, so semantically the same. All such hypothesis having same semantic is counted as 1. So we can have total number of combinations as below.

1 (hypothesis with one or more 0) + $4 \times 3 \times 3 \times 3 \times 3 \times 3$ (add ? to each attribute) = 973 semantically distinct hypotheses [Only one more value for attributes: ?, and one hypothesis representing empty set of instances.]

General-to-Specific Ordering of Hypotheses

Consider two hypotheses, **-**

h1 = (Sunny, ?, ?, Strong, ?, ?)

h2 = (Sunny, ?, ?, ?, ?, ?)

The hypothesis h2 has fewer constraints on attributes than hypothesis h1.

The sets of instances that are classified positive by h1 will be less than sets of instances that are classified positive by h2 as h2 imposes fewer constraints on the instance.

Any instance classified positive by hl will also be classified positive by h2.

Therefore, we say that h2 is more general than hl. In reverse, we can also say that, h1 is more specific than h2.

General-to-Specific Ordering of Hypotheses

Hypothesis h1 and h2 classifies the instance x as positive can written as h1(x) = 1 and h2(x) = 1.

Now h2 is more general than h1, so it can be written as if h1(x) = 1 implies h2(x) = 1. In symbols .

$$h_2 \geq_g h_1$$

Definition for **more-general-than-or-equal-to** –

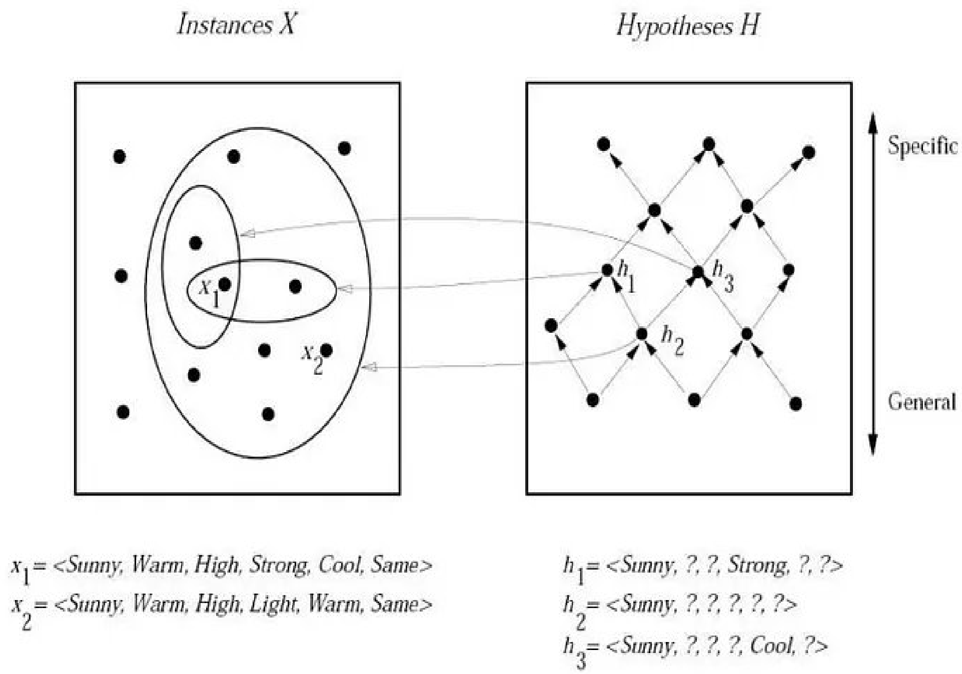
Definition: Let h_j and h_k be Boolean-valued functions defined over X. Then h_j is **more general-than-or-equal-to** h_k (written $h_j \geq_g h_k$) if and only if

$$(\forall x \in X)[h_k(x) = 1 \rightarrow h_j(x) = 1]$$

In similar way, **more-specific-than**, also be defined as below.

The h_j is **more-general-than** h_k ($h_j > h_k$) if and only if $h_j \geq_g h_k$ is true and $h_k \geq_g h_j$ is false. We also say h_k is **more-specific-than** h_j.

General-to-Specific Ordering of Hypotheses



- The box on the left represents the set X of all instances, the box on the right the set H of all hypotheses.
- Each hypothesis corresponds to some subset of X, that is, the subset of instances that it classifies positive. In the figure, there are 2 instances x1 and x2, and 3 hypothesis h1, h2 and h2.
- h1 classifies — x1, h2 classifies — x1 and x2, and h3 classifies — x1. This indicates, h2 is more-general-than h1 and h3.
- The arrows connecting hypotheses represent the more-general-than relation, with the arrow pointing toward the less general hypothesis.
- Note the subset of instances characterized by h2 subsumes the subset characterized by hl ,hence h2 is more-general–than h1

Many learning algorithms for concept learning organize the search through the hypothesis space by relying on a general-to-specific ordering of hypotheses.

Find-s: finding a maximally specific hypothesis

1.Initialize h to the most specific hypothesis in H.

2.For each positive training instance x

For each attribute constraint a_i in h

If the constraint a_i is satisfied by x

Then do nothing

Else replace a_i in h by the next more general constraint that is satisfied by x

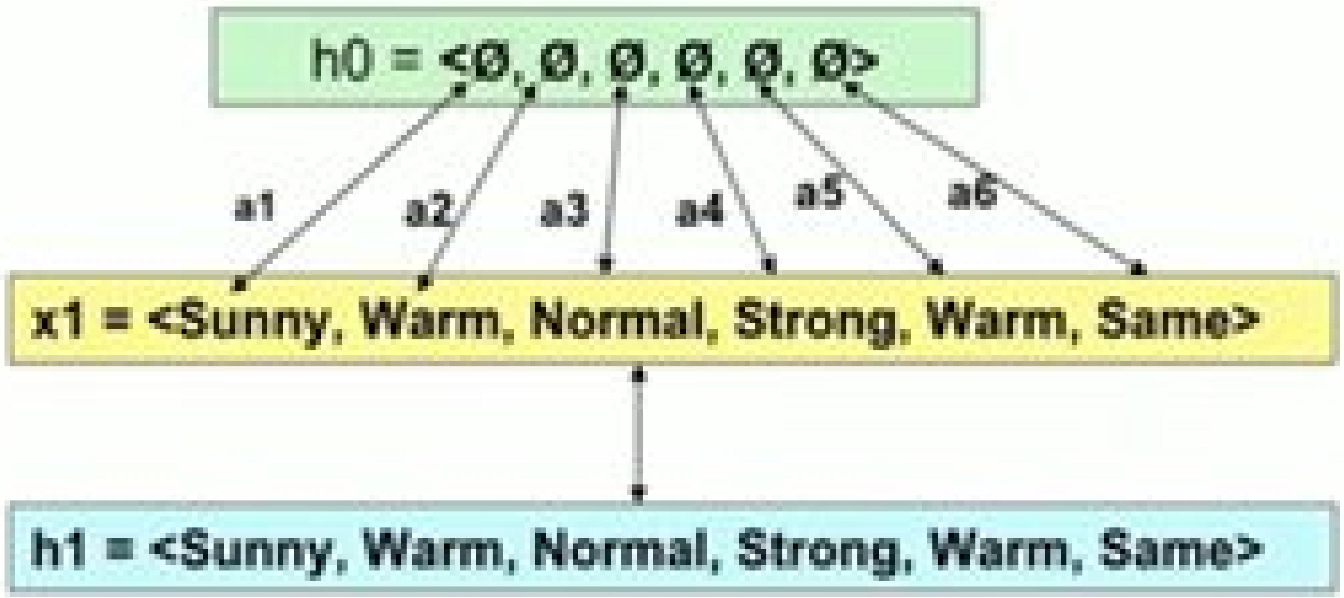
3. Output hypothesis h

Example:

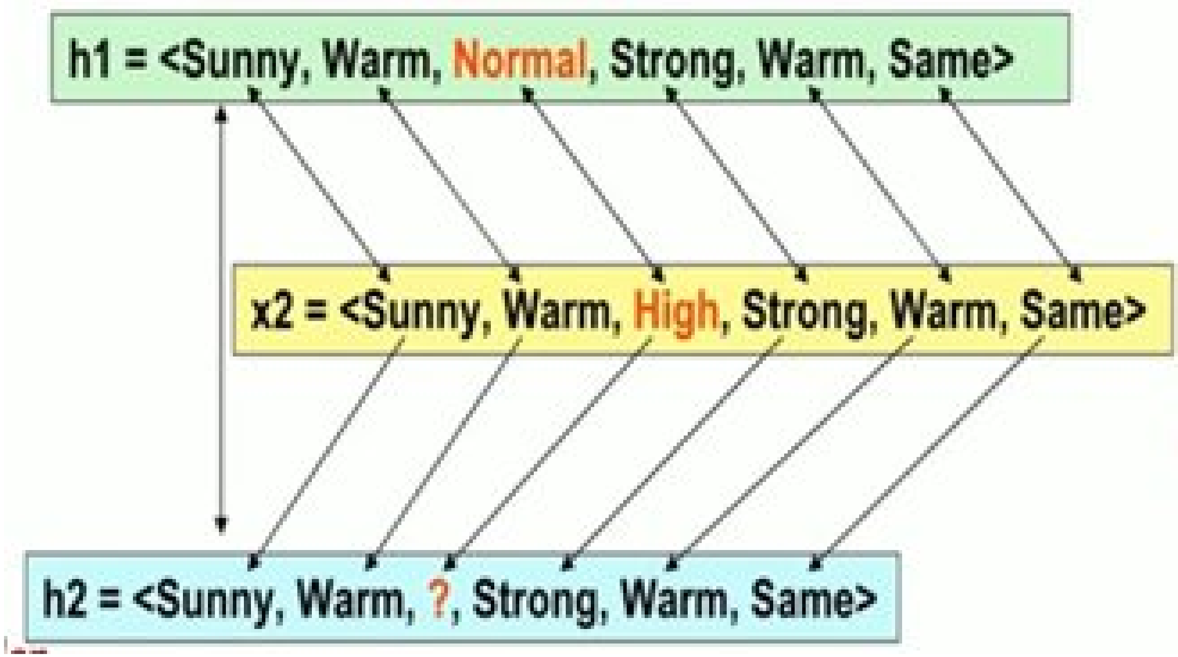
Assume the learner is given the sequence of training examples from the EnjoySport task

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Iteration 1


h0 = <Ø, Ø, Ø, Ø, Ø, Ø>
a1
a2
a3
a4
a5
a6
x1 = <Sunny, Warm, Normal, Strong, Warm, Same>
h1 = <Sunny, Warm, Normal, Strong, Warm, Same>

Iteration 2

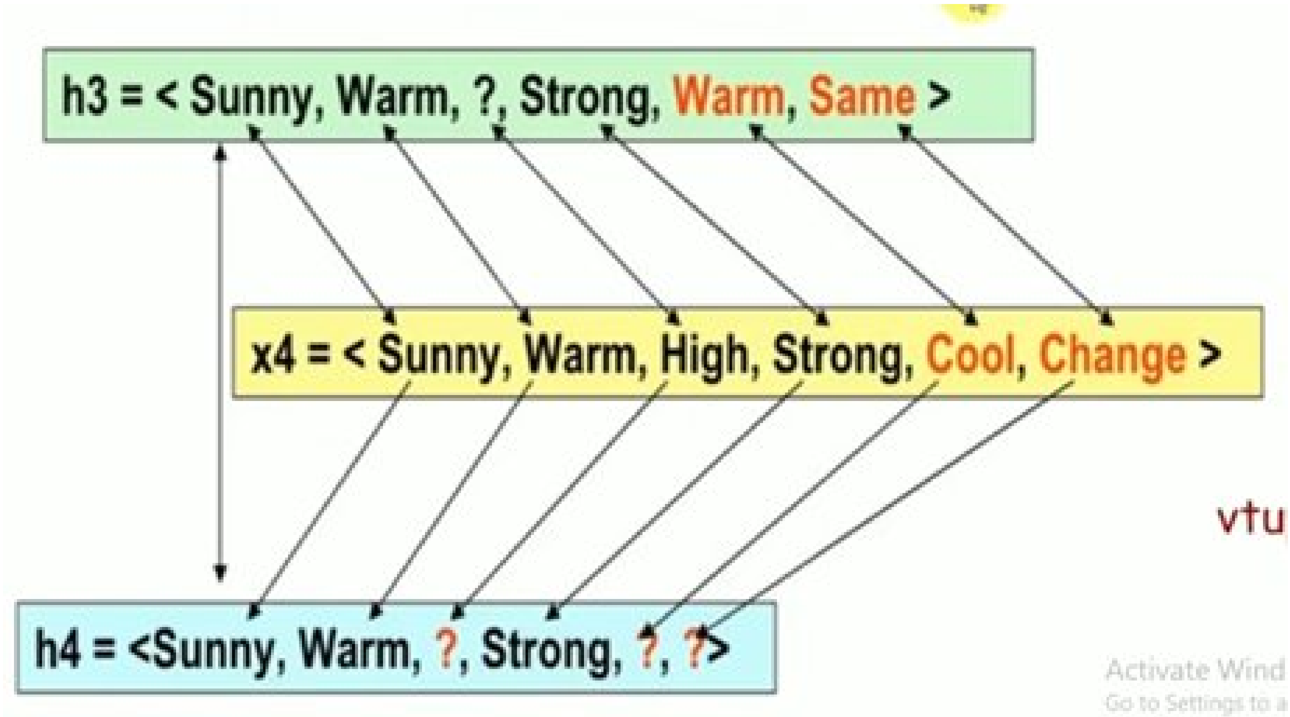

h1 = <Sunny, Warm, Normal, Strong, Warm, Same>
x2 = <Sunny, Warm, High, Strong, Warm, Same>
h2 = <Sunny, Warm, ?, Strong, Warm, Same>

Iteration 3

Ignore

h3 = <Sunny, Warm, ?, Strong, Warm, Same>

Iteration 4



h3 = < Sunny, Warm, ?, Strong, Warm, Same >
x4 = < Sunny, Warm, High, Strong, Cool, Change >
h4 = <Sunny, Warm, ?, Strong, ?, ?>
vtu
Activate Wind
Go to Settings to a

Find-s

1. Initialize h to the most specific hypothesis in H **h — (Ø, Ø, Ø, Ø, Ø, Ø)**

2. First training example

x1 = < Sunny, Warm, Normal, Strong ,Warm ,Same>, EnjoySport = +ve.

Observing the first training example, it is clear that hypothesis h is too specific. None of the "Ø" constraints in h are satisfied by this example, so each is replaced by the next more general constraint that fits the example

h1 = < Sunny, Warm, Normal, Strong ,Warm, Same>.

Find-s

3. Consider the second training example

x2 = < Sunny, Warm, High, Strong, Warm, Same>, EnjoySport = +ve.

The second training example forces the algorithm to further generalize h, this time substituting a “?” in place of any attribute value in h that is not satisfied by the new example.

Now h2 =< Sunny, Warm, ?, Strong, Warm, Same>

Find-s

4. Consider the third training example

x3 =< Rainy, Cold, High, Strong, Warm, Change>,EnjoySport = — ve.

The FIND-S algorithm simply ignores every negative example. So the hypothesis remain as before,

so **h3 =< Sunny, Warm, ?, Strong, Warm, Same>**

Find-s

5. Consider the fourth training example

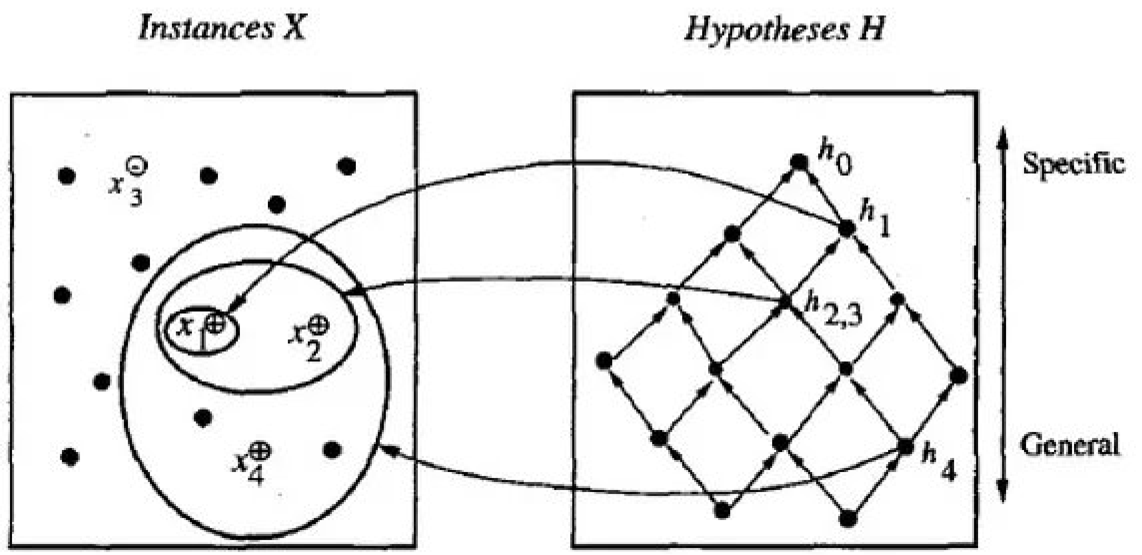
x4 =<Sunny,Warm,High,Strong, Cool,Change>, EnjoySport =+ve.

The fourth example leads to a further generalization of h as

h4 =< Sunny, Warm, ?, Strong, ?, ?>

6. So the final hypothesis is **< Sunny, Warm, ?, Strong, ?, ?>**

Find-s



x_1 = <Sunny Warm Normal Strong Warm Same>, +
x_2 = <Sunny Warm High Strong Warm Same>, +
x_3 = <Rainy Cold High Strong Warm Change>, -
x_4 = <Sunny Warm High Strong Cool Change>, +

h_0 = <Ø, Ø, Ø, Ø, Ø, Ø>
h_1 = <Sunny Warm Normal Strong Warm Same>
h_2 = <Sunny Warm ? Strong Warm Same>
h_3 = <Sunny Warm ? Strong Warm Same>
h_4 = <Sunny Warm ? Strong ? ? >

The key property of the FIND-S algorithm

- FIND-S is guaranteed to output the most specific hypothesis within H that is consistent with the positive training examples.
- FIND-S algorithm's final hypothesis will also be consistent with the negative examples provided the correct target concept is contained in H, and provided the training examples are correct.

Unanswered questions by FIND-S

1. **Has the learner converged to the correct target concept ?.** Although FIND-S will find a hypothesis consistent with the training data, it has no way to determine whether it has found the only hypothesis in H consistent with the data (i.e., the correct target concept), or whether there are many other consistent hypotheses as well.
2. **Why prefer the most specific hypothesis ?.** In case there are multiple hypotheses consistent with the training examples, FIND-S will find the most specific. It is unclear whether we should prefer this hypothesis over, say, the most general, or some other hypothesis of intermediate generality.

Unanswered questions by FIND-S

3. Are the training examples consistent ?. In most practical learning problems there is some chance that the training examples will contain at least some errors or noise. Such inconsistent sets of training examples can severely mislead FIND-S, given the fact that it ignores negative examples.

4. What if there are several maximally specific consistent hypotheses?. There can be several maximally specific hypotheses consistent with the data. Find S finds only one.

Consistent Hypothesis

A hypothesis h is consistent with a set of training examples D if and only if h(x) = c(x) for each example (x, c(x)) in D.

$$Consistent\ (h, D) \equiv (\forall \langle x, c(x)\rangle \in D)\ h(x) = c(x))$$

Difference between consistent and satisfies

A training example x is said to satisfy hypothesis h when h(x) = 1, regardless of whether x is a positive or negative example of the target concept.

An example x is said to consistent with hypothesis h iff h(x) = c(x)

Consistent Hypothesis- Example 1

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Consistent Hypothesis- Example 1

We found the hypothesis h 1= **< Sunny, Warm, ?, Strong, ?, ?>.** Now for each example (x, c(x)) in D, we will evaluate h(x) equals c(x).

1. (<Sunny, Warm, Normal, Strong, Warm, Same>,<yes>) → h(x)=c(x)
2. (<Sunny, Warm, High, Strong, Warm, Same>,<yes>) → h(x)=c(x)
3. (<Rainy, Cold, High, Strong, Warm, Change>,<No>) → h(x)=c(x)
4. (<Sunny, Warm, High, Strong, Cool, Change>,<yes>) → h(x)=c(x)

Now we can say, hypothesis h1 is consistent with a set of training examples D

Qn: Hypothesis h2 = < ?, ?, ?, Strong, ?, ?>, is this hypothesis consistent with set of training example D ?

In case of training example (3), h(x) != c(x). So hypothesis h1 is not consistent with D.

Qn: We have a hypothesis h3= < ?, Warm, ?, Strong, ?, ?>, is this hypothesis consistent with set of training example D ?

All the training examples hold h(x) = c(x). So hypothesis h2 is consistent with D.

Consistent Hypothesis- Example 2

Example	Citations	Size	InLibrary	Price	Editions	Buy
1	Some	Small	No	Affordable	One	No
2	Many	Big	No	Expensive	Many	Yes

h1 = (?, ?, No, ?, Many) – Yes —- is a consistent hypothesis

h2 = (?, ?, No, ?, ?) – yes —- is inconsistent hypothesis

Check consistent/not- identify the violating example(s)

1. x1=(<Sunny,Hot,Normal,Strong,Warm,Same>,<Yes>) → h(x)=c(x)
2. x2=(<Sunny,Cool,High,Weak,Warm,Change>,<No>) → h(x)=c(x)
3. x3=(<Rainy,Mild,Normal,Strong,Cool,Same>,<Yes>) → h(x)=c(x)
4. x4=(<Overcast,Hot,High,Weak,Warm,Same>,<Yes>) → h(x)=c(x)

h1 = <Sunny, ?, ?, ?, ?, ?>

h2 = < ?, ?, ?, Strong, ?, ?>

h3 = <Sunny, Hot, ?, ?, ?, ?>

h4 = < ?, Cool, ?, ?, ?, ?>

h5 = < ?, ?, Normal, ?, ?, ?>

h6 = <Overcast, ?, ?, ?, ?, ?>

h7 = < ?, ?, High, Weak, ?, ?>

h8 = <Rainy, ?, ?, Strong, ?, ?>

h9 = <Sunny, ?, Normal, ?, ?, ?>

h10 =< ?, Hot, ?, ?, Warm, ?>

Version space

The version space, denoted VS_H,D with respect to hypothesis space H and training examples D, is the subset of hypotheses from H consistent with the training examples in D

$$VS_{H,D} \equiv \{h \in H \mid Consistent(h, D)\}$$

In the previous example, we have two hypothesis from H and they are consistent with D.

h1=< Sunny, Warm, ?, Strong, ?, ?>

and h3=< ?, Warm, ?, Strong, ?, ?>

So this set of hypothesis { h1, h3} is called a Version Space.

List-Then-Eliminate algorithm

1. ***VersionSpace* c** *a* list containing every hypothesis in H
2. For each training example, ***(x, c(x))***

   remove from ***VersionSpace*** any hypothesis ***h*** for which $h(x) \neq c(x)$
3. Output the list of hypotheses in ***VersionSpace***

List-Then-Eliminate algorithm- Example1

Example: List-Then-Eliminate algorithm

F1 – > A, B

F2 – > X, Y

Instance Space: (A, X), (A, Y), (B, X), (B, Y) – **4 Examples**

Hypothesis Space: (A, X), (A, Y), (A, **ø**), (A, ?), (B, X), (B, Y), (B, **ø**), (B, ?), (**ø**, X), (**ø**, Y), (**ø**, **ø**), (**ø**, ?), (**?**, X), (**?**, Y), (**?**, **ø**), (**?**, ?) – **16 Hypothesis**

Semantically Distinct Hypothesis : (A, X), (A, Y), (A, ?), (B, X), (B, Y), (B, ?), (**?**, X), (**?**, Y (**?**, ?), (**ø**, **ø**) – **10**

List-Then-Eliminate algorithm- Example1

Solution:

Version Space: (A, X), (A, Y), (A, ?), (B, X), (B, Y), (B, ?), (?, X), (?, Y) (?, ?), (**ø**, **ø**),

Training Instances

F1	F2	Target
A	X	Yes
A	Y	Yes

Consistent Hypothesis are: (A, ?), (?, ?)

Problems: List-Then-Eliminate algorithm

1. The hypothesis space must be finite
2. Enumeration of all the hypothesis, in practice it is not feasible.thereforer inefficient

List-Then-Eliminate algorithm- Example2

For the enjoysport training examples D, we can output the below list of hypothesis which are consistent with D. In other words, the below list of hypothesis is a version space.

h1	Sunny	?	?	?	?	?
h2	?	Warm	?	?	?	?
h3	Sunny	?	?	Strong	?	?
h4	Sunny	Warm	?	?	?	?
h5	?	Warm	?	Strong	?	?
h6	Sunny	Warm	?	Strong	?	?

List-Then-Eliminate algorithm- Example

In the list of hypothesis, there are two extremes representing general (h1 and h2) and specific (h6) hypothesis. Lets define these 2 extremes as general boundary G and specific boundary S.

General boundary G

The **general boundary** G, with respect to hypothesis space H and training data D, is the set of maximally general members of H consistent with D.

Specific boundary S

The **specific boundary** S, with respect to hypothesis space H and training data D, is the set of minimally general (i.e., maximally specific) members of H consistent with D.

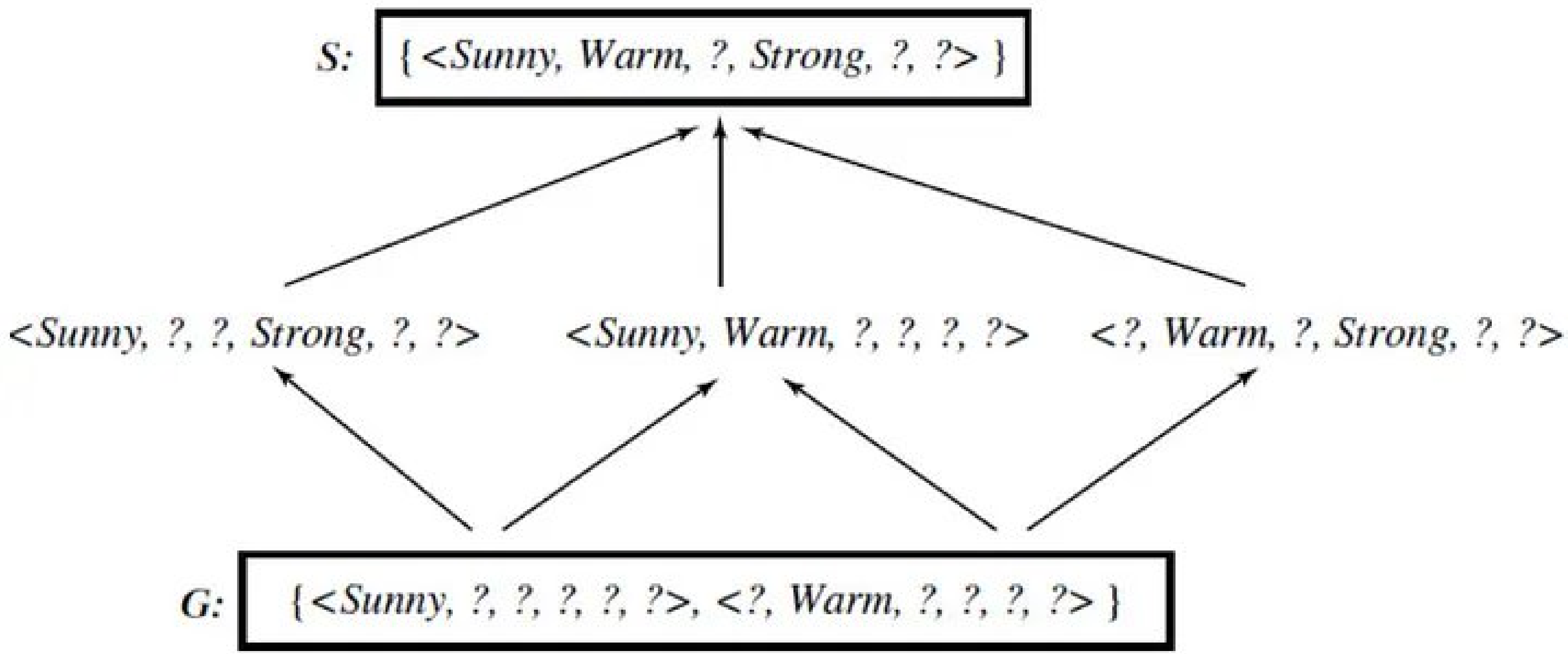
Version Space representation theorem

Let X be an arbitrary set of instances and Let H be a set of Boolean-valued hypotheses defined over X. Let c: X →{O, 1} be an arbitrary target concept defined over X, and let D be an arbitrary set of training examples {(x, c(x))). For all X, H, c, and D such that S and G are well defined,

$$VS_{H,D}=\{h \in H \mid (\exists s \in S)\ (\exists g \in G)\ (g \geq_g h \geq_g s)\}$$

The below figure shows the version space for enjoysport concept learning including both general and specific boundary sets.

Version Space representation theorem



From Version Space representation theorem, there exist a hypothesis s belongs to specific boundary set S, and there exist a hypothesis g belongs to general boundary set G, a hypothesis h belongs hypothesis space H holding the relationship "g is **more-general-than-or-equal-to** h and h is **more-general-than-or-equal-to** s" forms a version space.

Candidate Elimination algorithm

Initialize G to the set of maximally general hypotheses in H

Initialize S to the set of maximally specific hypotheses in H

For each training example d, do

- If d is a positive example
  - Remove from G any hypothesis inconsistent with d
  - For each hypothesis s in S that is not consistent with d
    - Remove s from S
    - Add to S all minimal generalizations h of s such that
      - h is consistent with d, and some member of G is more general than h
    - Remove from S any hypothesis that is more general than another hypothesis in S

- If d is a negative example
  - Remove from S any hypothesis inconsistent with d
  - For each hypothesis g in G that is not consistent with d
    - Remove g from G
    - Add to G all minimal specializations h of g such that
      - h is consistent with d, and some member of S is more specific than h
    - Remove from G any hypothesis that is less general than another hypothesis in G

Candidate Elimination algorithm

- The Candidate-Elimination algorithm computes the version space containing all hypotheses from H that are consistent with an observed sequence of training examples.
- It begins by initializing the version space to the set of all hypotheses in H; that is, by initializing the G boundary set to contain the most general hypothesis in H as

  G0 ← { <?, ?, ?, ?, ?, ?> }

  and initializing the S boundary set to contain the most specific hypothesis as

  S0 ← { <0, 0, 0, 0, 0, 0> }

Candidate Elimination algorithm

- These two boundary sets delimit the entire hypothesis space, because every other hypothesis in H is both more general than S0 and more specific than G0.
- As each training example is considered, the S and G boundary sets are generalized and specialized, respectively, to eliminate from the version space any hypotheses found inconsistent with the new training example.
- After all examples have been processed, the computed version space contains all the hypotheses consistent with these examples and only these hypotheses.

Candidate Elimination algorithm- Example

Training examples D

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Candidate Elimination algorithm- Example

Write most specific and generic boundaries

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

S_0: $\boxed{\langle \varnothing, \varnothing, \varnothing, \varnothing, \varnothing. \varnothing \rangle}$

G_0: $\boxed{\langle ?, ?, ?, ?, ?, ? \rangle}$

Iteration 1

- Consider first example- +ve.
- First goto generic and check hypo at generic boundary is consistent/not
- if consistent

    Retain it

- else

    Write next general

- In this case,consistent, therefore retain

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

S_0: $\boxed{\langle \varnothing, \varnothing, \varnothing, \varnothing, \varnothing. \varnothing \rangle}$

G_0: G_1: $\boxed{\langle ?, ?, ?, ?, ?, ? \rangle}$

Iteration 1

Then goto specific->

if not consistent,write next specific

Here not consistent, therefore write next specific

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

S_0: ⟨∅, ∅, ∅, ∅, ∅. ∅⟩

S_1: ⟨*Sunny, Warm, Normal, Strong, Warm, Same*⟩

G_0: G_1: ⟨?, ?, ?, ?, ?, ?⟩

Iteration 2

second example, +ve

first goto generic->Consistent

Retain

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

S_0: $\boxed{\langle \varnothing, \varnothing, \varnothing, \varnothing, \varnothing. \varnothing \rangle}$

S_1: $\boxed{\langle Sunny, Warm, Normal, Strong, Warm, Same \rangle}$

G_0: G_1: G_2 $\boxed{\langle ?, ?, ?, ?, ?, ? \rangle}$

Iteration 2

Then goto specific->Inconsistent

write new specific

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

S_0: $\boxed{\langle \varnothing, \varnothing, \varnothing, \varnothing, \varnothing. \varnothing \rangle}$

S_1: $\boxed{\langle \text{Sunny, Warm, Normal, Strong, Warm, Same} \rangle}$

S_2: $\boxed{\langle \text{Sunny, Warm, ?, Strong, Warm, Same} \rangle}$

G_0: G_1: G_2: $\boxed{\langle ?, ?, ?, ?, ?, ? \rangle}$

Iteration 3

Third example, -ve

First goto specific->consistent(No and expected also No). Therefore retain

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

S_0: $\langle \varnothing, \varnothing, \varnothing, \varnothing, \varnothing. \varnothing \rangle$

S_1: ⟨*Sunny,Warm, Normal, Strong, Warm, Same*⟩

S_2: S_3: ⟨*Sunny,Warm, ?, Strong, Warm, Same*⟩

G_0: G_1: G_2: ⟨?, ?, ?, ?, ?, ?⟩

Iteration 3

goto generic->inconsistent

write all hypothesis which are consistent till now

replace each ? mark with opposite of the value in the example

check whether all of these are consistent/not with all examples till now(upto 3rd eg)

if not, remove them (in red color)

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

S_0: ⟨∅, ∅, ∅, ∅, ∅. ∅⟩

S_1: ⟨*Sunny, Warm, Normal, Strong, Warm, Same*⟩

S_2: S_3: ⟨*Sunny, Warm, ?, Strong, Warm, Same*⟩

G_3: ⟨*Sunny,?,?,?,?,?*⟩ ⟨*?,Warm,?,?,?,?*⟩ ⟨*?,?,Normal,?,?,?*⟩ ⟨*?, ?,?,?,Cool,?*⟩ ⟨*?,?,?,?,?,Same*⟩

G_0 G_1 G_2: ⟨?, ?, ?, ?, ?, ?⟩

Iteration 4

Last example- +ve

First goto generic

retain all consistent

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

S_0: ⟨∅, ∅, ∅, ∅, ∅. ∅⟩

S_1: ⟨*Sunny,Warm, Normal, Strong, Warm, Same*⟩

S_2: S_3: ⟨*Sunny,Warm, ?, Strong, Warm, Same*⟩

G_4: ⟨*Sunny, ?, ?, ?, ?, ?*⟩ ⟨*?, Warm, ?, ?, ?, ?*⟩

G_3: ⟨*Sunny,?,?,?,?,?*⟩ ⟨*?,Warm,?,?,?,?*⟩ ⟨*?,?,Normal,?,?,?*⟩ ⟨*?, ?,?,?,Cool,?*⟩ ⟨*?,?,?,?,?,Same*⟩

G_0 G_1 G_2: ⟨?, ?, ?, ?, ?, ?⟩

Iteration 4

Then goto specific->Inconsistent

replace with ? in the specific

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

S_0: ⟨∅, ∅, ∅, ∅, ∅. ∅⟩

S_1: ⟨*Sunny,Warm, Normal, Strong, Warm, Same*⟩

S_2: S_3: ⟨*Sunny,Warm, ?, Strong, Warm, Same*⟩

S_4 ⟨*Sunny, Warm, ?, Strong, ?, ?*⟩

G_4: ⟨*Sunny, ?, ?, ?, ?, ?*⟩ ⟨*?, Warm, ?, ?, ?, ?*⟩

G_3: ⟨*Sunny,?,?,?,?,?*⟩ ⟨*?,Warm,?,?,?,?*⟩ ⟨*?,?,Normal,?,?,?*⟩ ⟨*?, ?,?,?,Cool,?*⟩ ⟨*?,?,?,?,?,Same*⟩

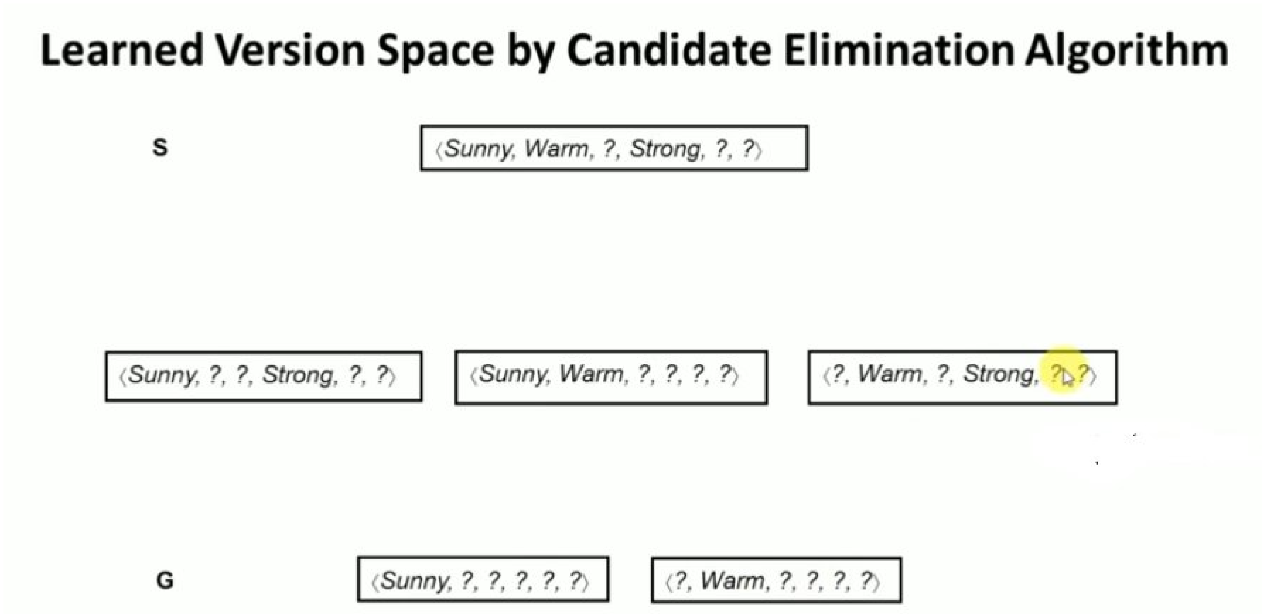
G_0 G_1 G_2: ⟨?, ?, ?, ?, ?, ?⟩

Learned Version Space by Candidate Elimination Algorithm

S4 and G4 are generic and specific boundaries

since more than one hypothesis,write some more possibilities

consider one from each and compare for the each mismatch and replce ? with specific



Total consistent hypothesis for the given set=6